MAESTROS A UN CLIC

Documentación del proyecto

Plataforma web que conecta clientes con profesionales verificados de servicios del hogar en Medellín

Integrantes

- Juan Pablo Castaño
- Brayan Arenas

Institución Universitaria Pascual Bravo
Tecnología en Desarrollo de Software · Sexto semestre
Ingeniería de Software 2

Docente: Maira Perez

Medellín, Octubre de 2026

Documento del proyecto

> Versión revisada del entregable del proyecto de aula (13/04/2026). Se corrigió la numeración, se actualizaron los datos técnicos y legales, y el alcance refleja lo que el sistema hace hoy. Los requisitos, los diagramas, las pruebas, el despliegue y la seguridad tienen documentos propios, enlazados en cada sección.

1. Introducción

¿Alguna vez has necesitado instalar cajones nuevos en la cocina y no sabías quién podía hacerlo? Muchas personas enfrentan situaciones parecidas con tareas del hogar: reparaciones, renovaciones, instalaciones y diseño. El problema es que siempre hay que buscar en lugares distintos o preguntar a conocidos para encontrar a alguien que haga el trabajo. **Maestros a un clic** centraliza a esos profesionales, cerca de ti, para que solicites su servicio con un clic.

La solución es una plataforma web en la que:

- **Los profesionales** se registran, eligen sus especialidades, publican sus servicios con precio y duración estimada, muestran un portafolio de trabajos y definen su horario y sus tiempos de descanso. Para generar confianza, pasan por una verificación de identidad: suben documento, selfie y certificados, que revisa un administrador.
- **Los clientes** se registran, buscan profesionales por categoría, nombre, precio y cercanía, y reservan una hora libre real de la agenda del profesional. Para reservar también deben estar verificados.
- **Las dos partes** se comunican por un chat dentro de cada reserva, reciben notificaciones en tiempo real de cada cambio y, al terminar el servicio, se califican mutuamente de 1 a 5 estrellas. Una calificación baja genera una alerta para que el administrador revise el caso.

2. Planteamiento del problema

En Medellín, encontrar un profesional confiable para el hogar es difícil. Las alternativas actuales son:

- **Empresas que tercerizan profesionales**: tienen costos altos y el cliente no elige quién hace el trabajo.
- **El voz a voz**: depende de los contactos de cada persona y los resultados no siempre son los esperados.
- **Los grupos de redes sociales**: no hay ninguna validación de identidad ni de experiencia, y es común que haya estafas.

Además, coordinar horarios por mensajes es lento y propenso a cruces: el profesional acepta dos trabajos a la misma hora o no deja tiempo para desplazarse.

3. Justificación

La plataforma busca dar **credibilidad** al profesional y **tranquilidad** al cliente, para que este vuelva a usarla. Lo logra con cuatro mecanismos:

1. **Verificación de identidad** de clientes y profesionales, con documentos y certificados revisados por un administrador.
2. **Calificación recíproca y visible**: el cliente califica al profesional y el profesional al cliente, así otros profesionales pueden evaluar a un cliente antes de aceptar un trabajo.
3. **Agenda real**: el profesional define su horario, sus bloqueos y un tiempo de descanso entre servicios. El cliente solo ve y reserva horas libres, y el sistema impide que dos servicios se crucen.
4. **Trazabilidad**: cada cambio de estado de una reserva queda registrado, con quién lo hizo, cuándo y por qué.

El flujo principal es este: el profesional se registra y se verifica, y publica sus servicios. El cliente busca un profesional y envía una solicitud para una hora libre. El profesional la acepta o la rechaza; el cliente confirma. El profesional inicia y finaliza el servicio, y las dos partes se califican.

4. Objetivos

4.1 Objetivo general

Desarrollar una plataforma web de servicios profesionales y oficios que conecte a especialistas en mantenimiento, infraestructura y diseño del hogar con clientes de Medellín, y que facilite una contratación transparente, verificada y sin intermediarios.

4.2 Objetivos específicos

1. Construir un sistema de **perfiles profesionales** en el que el prestador muestre evidencia fotográfica de su trabajo, sus especialidades, sus servicios con precio y duración, y su ubicación.
2. Implementar un **motor de reservas** que calcule la disponibilidad real del profesional (horario, bloqueos, descanso y reservas existentes), evite cruces de horario y lleve cada reserva por un ciclo de vida auditable: solicitud, aceptación, confirmación, inicio y finalización.
3. Establecer un **ecosistema de confianza** basado en verificación de identidad, ubicación por comunas de Medellín y calificación recíproca.
4. Ofrecer **comunicación en tiempo real** entre las partes: chat por reserva, estado en línea y notificaciones.
5. Aplicar prácticas profesionales de ingeniería de software: arquitectura por capas y módulos, contrato de API documentado, pruebas automatizadas, integración continua y control de versiones con revisión por pull request.

5. Referentes del medio

Referente	Cómo funciona	Limitación frente a esta propuesta
Hombre en Casa Medellín	Página informativa; la solicitud se hace por WhatsApp y la empresa contacta a un profesional	Proceso manual. El cliente no elige al profesional ni ve su agenda
Multihelpers	Se escribe a la empresa, que asigna el profesional	El cliente no elige ni conoce el perfil del profesional
Hogaru	Servicio de aseo para hogares y empresas	Un solo nicho. La empresa asigna a la persona y no se ve su perfil

Diferencial de Maestros a un clic: el cliente elige al profesional viendo su perfil verificado, su portafolio, sus calificaciones y su agenda real. La reserva es directa y queda trazada de principio a fin.

6. Marco de referencia

- **Marco contextual.** La plataforma arranca en Medellín y usa la división administrativa en 16 comunas y 5 corregimientos para ubicar a clientes y profesionales y sugerir comunas vecinas.
- **Marco tecnológico.** Se enmarca en la economía colaborativa (*gig economy*). Usa una API REST, comunicación en tiempo real por WebSockets, una base de datos relacional y geolocalización.
- **Marco conceptual.** Es un *marketplace de servicios*: intermediación digital, validación de identidad (*KYC, Know Your Customer*) para reducir fraudes y reputación basada en calificación recíproca.

7. Metodología

7.1 Sustento teórico

El proyecto se fundamenta en **Scrum** (Schwaber y Sutherland, 2020): entregas iterativas e incrementales, colaboración del equipo y adaptación al cambio. Las historias de usuario, con criterios de aceptación verificables, forman el *product backlog*.

7.2 Cómo se aplicó

Práctica	Implementación en el proyecto
Backlog	Cada historia de usuario es un issue de GitHub con sus criterios de aceptación como lista verificable, su módulo y sus dependencias
Reparto del trabajo	Por módulos completos (backend y frontend de cada uno) entre dos desarrolladores *full stack*; cada uno es dueño de sus módulos
Fase 0 (base común)	Antes de repartir: esquema completo de la base de datos, contrato de la API, sistema de diseño, convenciones, CI y un módulo de referencia (favoritos)
Iteraciones	Una rama por historia, pull request revisado y fusión a `main` solo con la integración continua en verde
Definición de terminado	Cumple los criterios de aceptación, tiene pruebas automatizadas (caso feliz, validación y permisos), está documentado en el contrato de la API y fusionado en los dos repositorios
Calidad automática	En cada PR, la integración continua corre las pruebas (PHP 8.2 y 8.5), el formato (Pint, Prettier), el lint (ESLint, oxlint), los tipos (TypeScript), el build, el verificador de colores del sistema de diseño y la auditoría de dependencias. El check **Guardas** impide aflojar reglas sin aprobación

Práctica	Implementación en el proyecto
Trabajo con IA	Los dos desarrolladores usan asistentes de IA (Claude Code y Antigravity) que leen las mismas reglas del repositorio (`AGENTS.md`), para mantener un estilo único

Las etapas clásicas siguen presentes (planificación, diseño, desarrollo, pruebas, despliegue y mantenimiento), pero en ciclos cortos por historia.

8. Estudio de factibilidad

8.1 Viabilidad técnica

Todas las tecnologías son de código abierto, maduras y con amplia documentación.

Capa	Tecnología	Uso
Frontend	Vue 3, TypeScript, Vite, Tailwind CSS 4, Pinia y Pinia Colada	Aplicación web de una sola página (SPA), responsive
Backend	Laravel 12 (PHP 8.2+), Laravel Sanctum	API REST versionada (`/api/v1`) y autenticación por token
Tiempo real	Laravel Reverb (WebSockets) y Laravel Echo	Chat, estado en línea, notificaciones y agenda en vivo
Base de datos	MySQL 8	Base relacional con integridad referencial
Calidad	Pest, Vitest, ESLint, oxlint, Pint, Prettier, GitHub Actions	Pruebas automatizadas, estilo y CI
Despliegue previsto	AWS EC2 con Nginx, Supervisor y certificado TLS	Servidor único para API, cola, Reverb y archivos

La arquitectura detallada está en 08-decisiones-de-arquitectura.md y en los documentos `docs/arquitectura.md` de cada repositorio.

8.2 Viabilidad económica

Costos de infraestructura (estimados, primer año):

Concepto	Costo
Vue, Laravel, Reverb, MySQL Community Edition (licencias de código abierto: MIT y GPL)	$0
Servidor AWS EC2 (instancia pequeña) y disco	≈ 25 USD/mes
Dominio y certificado TLS (Let's Encrypt)	≈ 15 USD/año y $0
Correo transaccional (recuperación de contraseña)	Plan gratuito de un proveedor SMTP en la fase inicial

Costos de talento humano (rangos de referencia en Colombia, para una operación real):

Rol	Funciones principales	Dedicación	Costo aproximado
Líder de proyecto	Coordinar el equipo, cronograma y objetivos	Completa durante el desarrollo	$4–6 millones COP/mes
Desarrollador frontend (Vue)	Interfaz, componentes, integración con la API	Completa	$3–5 millones COP/mes
Desarrollador backend (Laravel)	Lógica de negocio, API, seguridad	Completa	$3–5 millones COP/mes
Administrador de base de datos	Esquema, optimización, respaldos	Parcial	$2–3 millones COP/mes
DevOps / nube	Servidores, despliegue, CI/CD, monitoreo	Parcial	$3–4 millones COP/mes
Diseño UI/UX	Prototipos, pruebas de usabilidad	Parcial	$2–3 millones COP/mes
QA	Pruebas funcionales, de seguridad y rendimiento	Parcial	$2–3 millones COP/mes
Seguridad / KYC	Verificación de identidad, protección de datos	Parcial	$3–4 millones COP/mes
Soporte	Atención a usuarios y profesionales	Completa tras el lanzamiento	$2–3 millones COP/mes

Modelo de ingresos posible: comisión por servicio completado o suscripción de los profesionales para destacar su perfil. El cliente ahorra frente a las empresas que tercerizan, porque contrata directamente.

8.3 Viabilidad de mercado

Hay una demanda clara por la dificultad de encontrar profesionales confiables, y los referentes actuales no dejan elegir al profesional ni ofrecen procesos automatizados. Las principales dificultades previstas son:

1. **Adopción inicial**: los profesionales pueden resistirse a la verificación, y los clientes pueden preferir los métodos tradicionales.
2. **Competencia** con plataformas ya conocidas.
3. **Percepción de la verificación** como invasiva o compleja.
4. **Arranque en frío de la reputación**: al inicio no habrá reseñas, y se necesitarán incentivos para las primeras calificaciones.
5. **Cumplimiento legal** en el tratamiento de datos personales.
6. **Costos de mercadeo** para posicionar la marca.
7. **Escalabilidad de la oferta**: si la demanda crece rápido, puede faltar disponibilidad de profesionales.

8.4 Viabilidad operativa

La plataforma es **autogestionable**. Clientes y profesionales administran sus perfiles, documentos, agendas y reservas. El administrador solo interviene en los casos críticos: aprobar la verificación de documentos, atender alertas por baja calificación y revisar reportes. Así se puede atender a muchos más usuarios sin aumentar el personal en la misma proporción. Las notificaciones, la agenda y los cambios de estado son automáticos.

8.5 Viabilidad legal y normativa

Norma	Aplicación en la plataforma
Ley 1581 de 2012 (protección de datos personales)	Tratamiento de nombres, ubicación, selfies, documentos y certificados con finalidad explícita, seguridad y derechos del titular. Ver 07-seguridad-y-privacidad.md
Decreto 1377 de 2013 (hoy compilado en el Decreto 1074 de 2015)	Autorización previa e informada para el tratamiento, y procedimientos de consulta, rectificación y supresión
Ley 1266 de 2008 (hábeas data financiero)	No aplica hoy: no se maneja información crediticia. Se tendría en cuenta si se integraran pagos o validaciones financieras
Ley 527 de 1999 (comercio electrónico)	Validez de los mensajes de datos: las solicitudes, aceptaciones y confirmaciones digitales quedan registradas con fecha y autor
Ley 1273 de 2009 (delitos informáticos)	Protección de la información y de los datos: control de acceso, almacenamiento privado de documentos y registro de cambios
CONPES 3854 de 2016 y 3995 de 2020 (política de seguridad y confianza digital)	Marco de buenas prácticas de seguridad digital que orienta los controles del sistema
ISO/IEC 27001 y 27018	Referentes de gestión de seguridad de la información y protección de datos en la nube para la operación
ISO/IEC 25010	Modelo de calidad de producto usado para formular los requisitos no funcionales (02-requisitos.md)
RGPD (Unión Europea)	Referente voluntario de buenas prácticas para una posible expansión

> **Corrección respecto al entregable original.** La Ley 1621 de 2013 regula las actividades de inteligencia y contrainteligencia del Estado, no la seguridad digital, así que se reemplazó por la Ley 1273 de 2009 y los documentos CONPES. MySQL no tiene licencias de Microsoft: la edición Community es de Oracle y se distribuye bajo GPL.

9. Alcance y limitaciones

9.1 Alcance

Incluido	Estado
Registro e inicio de sesión de clientes y profesionales, cierre de sesión y validación de datos	Implementado
Perfil profesional: descripción, especialidades, tarifa, ubicación, servicios con precio y duración, y portafolio	Implementado
Agenda: horario semanal, descanso configurable, bloqueos y cálculo de horas libres en tiempo real	Implementado
Ciclo de vida de la reserva: solicitar, aceptar o rechazar, confirmar, reprogramar, cancelar, iniciar y finalizar, con historial auditable	Implementado

Incluido	Estado
Chat por reserva con imágenes, estado en línea y eliminación de mensajes	Implementado
Notificaciones en tiempo real y centro de notificaciones	Implementado
Historiales, totales, servicios en proceso e ingresos del profesional	Implementado
Favoritos y búsqueda por nombre, categoría y precio	Implementado
Verificación de identidad (cargue de documentos y revisión del administrador), bloqueo de usuarios	Implementado
Calificación recíproca, comentarios, reportes y alertas por baja calificación	Implementado
Búsqueda por cercanía con distancia, comunas aledañas, "disponible ahora" y mapa	Implementado
Gestión de la cuenta: editar perfil, foto, contraseña, recuperación, eliminación con anonimización y cierre por inactividad	Implementado

Las 49 historias de usuario están terminadas.

El detalle por historia, con sus endpoints, pantallas y pruebas, está en 04-trazabilidad.md.

9.2 Fuera del alcance

- **Pagos en línea.** El precio se acuerda en la plataforma, pero el pago se hace por fuera.
- **Consulta automática de antecedentes con un proveedor externo.** No existe una API pública de la Policía o la Procuraduría que se pueda integrar. El administrador registra el resultado manualmente, y el modelo de datos queda listo para un proveedor futuro (decisión D10).
- **Aplicaciones móviles nativas.** La aplicación web es responsive y funciona en el navegador del celular.

9.3 Limitaciones

- La plataforma facilita la conexión y la verificación, pero no garantiza la calidad del trabajo del profesional.
- La verificación depende de la revisión humana de un administrador.
- En la fase inicial, la cobertura geográfica se limita a Medellín.
- Los costos de infraestructura en la nube pueden variar con la demanda.

10. Requisitos

Los 49 requisitos funcionales (historias de usuario con sus criterios de aceptación y su estado) y los requisitos no funcionales, medibles y con su forma de verificación, están en **02-requisitos.md**.

11. Análisis y diseño

- **Diagramas** (casos de uso, clases del dominio, entidad-relación, estados de la reserva, secuencias, componentes y despliegue), generados a partir del sistema actual: **03-diagramas.md**. Reemplazan a los diagramas PNG del entregable original, que eran previos al modelo definitivo.
- **Decisiones de arquitectura** y su justificación: **08-decisiones-de-arquitectura.md**.
- **Modelo de datos** completo: docs/modelo-de-datos.md.
- **Contrato de la API**: docs/api/endpoints.md, convenciones y la especificación OpenAPI exportada en docs/api/openapi.json.

12. Pruebas

La estrategia, las herramientas y el reporte de resultados están en **05-pruebas.md**.

13. Despliegue y operación

La guía para poner el sistema en producción y operarlo está en **06-despliegue-y-operacion.md**.

14. Seguridad y privacidad

Los controles implementados y el cumplimiento de la Ley 1581 de 2012 están en **07-seguridad-y-privacidad.md**.

15. Conclusiones

- El proyecto es **viable** técnica, económica, operativa y legalmente: usa tecnologías de código abierto maduras, un costo de infraestructura bajo y un modelo autogestionable.
- La propuesta se **diferencia** de los referentes porque el cliente elige al profesional viendo su perfil verificado, su reputación y su agenda real, y la reserva queda trazada de principio a fin.
- Construir primero una **base común** (esquema completo, contrato de API, sistema de diseño, convenciones y CI) permitió que dos personas trabajaran en paralelo por módulos sin conflictos y con un estilo uniforme.
- Las **pruebas automatizadas y la integración continua** en cada pull request detectan regresiones antes de llegar a `main`. Junto con los documentos de requisitos y trazabilidad, dejan evidencia verificable de que cada historia cumple sus criterios.
- La implementación inicial en Medellín permite validar el modelo antes de escalarlo a otras ciudades.

16. Glosario

Término	Significado
Cliente	Persona que busca y reserva servicios
Profesional	Persona que ofrece servicios; también "maestro"
Servicio	Lo que ofrece un profesional, con precio y duración estimada
Reserva	Solicitud de un cliente para un servicio en una fecha y hora; pasa por varios estados
Disponibilidad / horas libres	Horas en las que se puede reservar: el horario del profesional menos sus bloqueos y reservas, con su descanso
Descanso (*buffer*)	Tiempo que se bloquea antes y después de cada reserva para desplazarse o descansar
Verificación (KYC)	Revisión de la identidad del usuario a partir de sus documentos
API REST	Interfaz por la que el frontend se comunica con el backend mediante HTTP y JSON
WebSocket	Conexión permanente que permite enviar eventos en tiempo real
CI	Integración continua: verificaciones automáticas en cada cambio
PR	*Pull request*: propuesta de cambio que se revisa antes de unirla a la rama principal

17. Bibliografía

- Congreso de Colombia. (1999). *Ley 527 de 1999*, por medio de la cual se define y reglamenta el acceso y uso de los mensajes de datos, del comercio electrónico y de las firmas digitales.
- Congreso de Colombia. (2008). *Ley 1266 de 2008*, disposiciones generales del hábeas data.
- Congreso de Colombia. (2009). *Ley 1273 de 2009*, de la protección de la información y de los datos.
- Congreso de Colombia. (2012). *Ley Estatutaria 1581 de 2012*, disposiciones generales para la protección de datos personales.
- Presidencia de la República de Colombia. (2013). *Decreto 1377 de 2013*, reglamentario de la Ley 1581 de 2012.
- Departamento Nacional de Planeación. (2016). *Documento CONPES 3854: Política Nacional de Seguridad Digital*.
- Departamento Nacional de Planeación. (2020). *Documento CONPES 3995: Política Nacional de Confianza y Seguridad Digital*.
- International Organization for Standardization. (2022). *ISO/IEC 27001: Information security management systems*.
- International Organization for Standardization. (2023). *ISO/IEC 25010: Product quality model*.
- OWASP Foundation. (2021). *OWASP Top 10* y *Application Security Verification Standard (ASVS) 4.0*.
- Parlamento Europeo y Consejo. (2016). *Reglamento (UE) 2016/679 (RGPD)*.
- Schwaber, K. y Sutherland, J. (2020). *The Scrum Guide*.
- Sommerville, I. (2016). *Software Engineering* (10.ª ed.). Pearson.
- Documentación oficial de Laravel 12, Vue 3 y Tailwind CSS 4 (laravel.com, vuejs.org, tailwindcss.com).

Requisitos

Requisitos funcionales (historias de usuario) y no funcionales del sistema. La fuente de verdad del estado de cada historia es su issue en GitHub; este documento muestra el estado al momento de generarlo.

1. Actores

Actor	Descripción
Visitante	Persona sin sesión: explora la búsqueda y los perfiles públicos
Cliente	Busca profesionales, reserva servicios, chatea y califica. Debe estar verificado para reservar
Profesional	Publica servicios, gestiona su agenda, responde solicitudes, presta el servicio y califica al cliente
Administrador	Verifica documentos, atiende reportes y alertas, y bloquea usuarios
Sistema	Calcula disponibilidad, emite notificaciones y eventos en tiempo real, y genera alertas automáticas

2. Requisitos funcionales

Las 49 historias de usuario del proyecto (no existe la HU015: la numeración del documento original la omite). Estado a la fecha de este documento: **las 49 están hechas**. Cada una es un issue en GitHub, donde se marcan sus criterios.

2.1 Resumen

HU	Nombre	Módulo	Responsable	Estado
HU001	Inicio de sesión	auth	LordCasta	✅ Hecha
HU002	Búsqueda por geolocalización	búsqueda	barenas12	✅ Hecha
HU003	Gestión de disponibilidad y agenda	agenda	LordCasta	✅ Hecha
HU004	Solicitud y aceptación de servicio	reservas	LordCasta	✅ Hecha
HU005	Sistema de calificación recíproca	calificaciones	barenas12	✅ Hecha
HU006	Registro de cliente	auth	LordCasta	✅ Hecha
HU007	Registro de profesional	auth	LordCasta	✅ Hecha
HU008	Cargue de documentos	verificación-admin	barenas12	✅ Hecha
HU009	Edición de perfil profesional	perfil-profesional	LordCasta	✅ Hecha
HU010	Filtros y ordenamiento de profesionales	búsqueda	barenas12	✅ Hecha
HU011	Chat y confirmación de detalles del servicio	chat-notificaciones	LordCasta	✅ Hecha
HU012	Cancelación y reprogramación de reserva	reservas	LordCasta	✅ Hecha
HU013	Validación de identidad y antecedentes	verificación-admin	barenas12	✅ Hecha
HU014	Revisión y aprobación de documentos del profesional	verificación-admin	barenas12	✅ Hecha
HU016	Historial de servicios del cliente	reservas	LordCasta	✅ Hecha
HU017	Historial de servicios del profesional	reservas	LordCasta	✅ Hecha
HU018	Notificaciones del sistema	chat-notificaciones	LordCasta	✅ Hecha
HU019	Cambio de contraseña	cuenta	barenas12	✅ Hecha
HU020	Recuperación de contraseña	cuenta	barenas12	✅ Hecha
HU021	Cerrar sesión	auth	LordCasta	✅ Hecha
HU022	Registro de ubicación manual	cuenta	barenas12	✅ Hecha
HU023	Eliminación de cuenta	cuenta	barenas12	✅ Hecha
HU024	Reporte de usuarios	calificaciones	barenas12	✅ Hecha

HU	Nombre	Módulo	Responsable	Estado
HU025	Visualización de disponibilidad del profesional	agenda	LordCasta	✅ Hecha
HU026	Estado del servicio	reservas	LordCasta	✅ Hecha
HU027	Confirmación de inicio de servicio	reservas	LordCasta	✅ Hecha
HU028	Confirmación de finalización de servicio	reservas	LordCasta	✅ Hecha
HU029	Visualización de comentarios	calificaciones	barenas12	✅ Hecha
HU030	Filtro por disponibilidad inmediata	búsqueda	barenas12	✅ Hecha
HU031	Edición de perfil del cliente	cuenta	barenas12	✅ Hecha
HU032	Subida de foto de perfil	cuenta	barenas12	✅ Hecha
HU033	Visualización de servicios por categoría	búsqueda	barenas12	✅ Hecha
HU034	Guardar profesionales favoritos	favoritos	LordCasta	✅ Hecha
HU035	Visualización de favoritos	favoritos	LordCasta	✅ Hecha
HU036	Confirmación de reserva por el cliente	reservas	LordCasta	✅ Hecha
HU037	Visualización de ingresos del profesional	perfil-profesional	LordCasta	✅ Hecha
HU038	Bloqueo de usuarios problemáticos	verificación-admin	barenas12	✅ Hecha
HU039	Búsqueda por nombre	búsqueda	barenas12	✅ Hecha
HU040	Visualización de distancia	búsqueda	barenas12	✅ Hecha
HU041	Indicador de estado en línea	chat-notificaciones	LordCasta	✅ Hecha
HU042	Adjuntar archivos en el chat	chat-notificaciones	LordCasta	✅ Hecha
HU043	Eliminación de mensajes	chat-notificaciones	LordCasta	✅ Hecha
HU044	Visualización de perfil del cliente	calificaciones	barenas12	✅ Hecha
HU045	Alerta por baja calificación	calificaciones	barenas12	✅ Hecha
HU046	Ordenamiento por precio	búsqueda	LordCasta	✅ Hecha
HU047	Visualización de tiempo estimado	perfil-profesional	LordCasta	✅ Hecha
HU048	Validación de datos obligatorios	auth	LordCasta	✅ Hecha
HU049	Visualización de servicios en proceso	reservas	LordCasta	✅ Hecha
HU050	Cierre de sesión por inactividad	cuenta	barenas12	✅ Hecha

2.2 Detalle

HU001 · Inicio de sesión

Yo como cliente, quiero iniciar sesión en el sistema, para validar los datos de mi cuenta

Criterios de aceptación:

1. El sistema valida las credenciales
2. El sistema muestra error si son incorrectas
3. Redirige al panel principal

Estado: ✅ Hecha

HU002 · Búsqueda por geolocalización

Yo como cliente, quiero buscar profesionales en mi comuna mediante un mapa o listado, para contratar un servicio rápido y eficiente cerca de mi ubicación.

Criterios de aceptación:

1. El sistema solicita permisos de ubicación al usuario.
2. El buscador filtra profesionales activos dentro del radio de acción configurado.
3. Se despliega una lista ordenada por proximidad (km).

4. Si no hay profesionales en la comuna, el sistema sugiere los más cercanos en comunas aledañas.

Estado: ✅ Hecha

HU003 · Gestión de disponibilidad y agenda

Yo como profesional, quiero configurar mis días y horas de trabajo en un calendario, para evitar que los clientes reserven en momentos donde no tengo disponibilidad.

Criterios de aceptación:

1. El profesional puede marcar bloques de horas como "ocupado" o "disponible".
2. El sistema bloquea automáticamente un tiempo adicional (configurable) después de cada servicio para descanso.
3. Los cambios en la agenda se reflejan en tiempo real para los clientes.
4. El calendario impide que se solapen dos servicios distintos.

Estado: ✅ Hecha

HU004 · Solicitud y aceptación de servicio

Yo como cliente, quiero solicitar una reserva seleccionando una hora disponible del profesional, para concretar la contratación sin necesidad de intermediarios.

Criterios de aceptación:

1. El cliente visualiza la disponibilidad real del profesional.
2. Al enviar la solicitud, el profesional recibe una alerta en su panel.
3. El profesional tiene la opción de "Aceptar" o "Rechazar" la solicitud.
4. El estado de la reserva cambia a "Confirmado" solo si el profesional acepta.

Estado: ✅ Hecha

HU005 · Sistema de calificación recíproca

Yo como usuario (cliente o profesional), quiero calificar mi experiencia con la otra parte tras finalizar el servicio, para mantener la reputación y calidad dentro de la plataforma.

Criterios de aceptación:

1. La opción de calificar solo se habilita una vez que el servicio ha sido marcado como "Completado".
2. El sistema permite una calificación de 1 a 5 estrellas.
3. Se permite dejar un comentario opcional sobre el servicio o la atención.
4. El promedio de estrellas se actualiza automáticamente en el perfil del usuario calificado.

Estado: ✅ Hecha

HU006 · Registro de cliente

Yo como cliente, quiero registrarme en la plataforma con mi correo y contraseña elegida., para acceder al sistema.

Criterios de aceptación:

1. El correo debe ser válido
2. Se debe validar la existencia del correo
3. La contraseña debe ser mayor a 5 caracteres
4. Me debe permitir iniciar sesión inmediatamente después de crear el usuario

Estado: ✅ Hecha

HU007 · Registro de profesional

Yo como profesional., quiero registrarme en la plataforma con mi correo y contraseña elegida., para acceder al sistema y poder cargar mis documentos.

Criterios de aceptación:

1. El correo debe ser válido
2. Se debe validar la existencia del correo
3. La contraseña debe ser mayor a 5 caracteres

4. Me debe permitir iniciar sesión inmediatamente después de crear el usuario

Estado: ✅ Hecha

HU008 · Cargue de documentos

Yo como profesional., quiero después de registrarme, subir mis documentos como certificaciones, fotos, diplomas requeridos., para poder que validen mis documentos y ser profesional verificado.

Criterios de aceptación:

1. El sistema permite cargar múltiples archivos (imágenes y PDF) por categoría (certificados, documento de identidad, evidencias de trabajos).
2. El sistema valida formato y tamaño máximo de archivo antes de subirlo.
3. El sistema solicita como mínimo: documento de identidad, selfie y al menos un certificado o evidencia de trabajo.
4. Al finalizar el cargue, el estado del perfil queda en "Pendiente de verificación" y no puede ofrecer servicios hasta ser aprobado.
5. El profesional puede ver el estado de revisión (Pendiente, Aprobado, Rechazado) y el motivo si es rechazado.
6. El profesional puede reemplazar documentos rechazados y reenviar a verificación.

Estado: ✅ Hecha

HU009 · Edición de perfil profesional

Yo como profesional, quiero editar mi perfil (descripción, especialidades, tarifa por hora, ubicación, agenda y portafolio), para mantener mi información actualizada y atraer más clientes.

Criterios de aceptación:

1. El sistema permite actualizar campos básicos del perfil sin perder información previamente aprobada.
2. El sistema valida que la tarifa por hora sea un valor numérico mayor a 0.
3. El sistema permite agregar o quitar especialidades dentro del catálogo de servicios.
4. Los cambios de ubicación actualizan la zona de cobertura para búsquedas por geolocalización.
5. Las fotos del portafolio se muestran en una galería dentro del perfil.

Estado: ✅ Hecha

HU010 · Filtros y ordenamiento de profesionales

Yo como cliente, quiero filtrar y ordenar profesionales por servicio, precio, calificación y distancia, para encontrar la mejor opción según mis necesidades.

Criterios de aceptación:

1. El cliente puede filtrar por categoría de servicio y subcategoría.
2. El cliente puede definir un rango de precio por hora.
3. El cliente puede ordenar por calificación (descendente) y por distancia (ascendente).
4. Los resultados muestran información clave: nombre, calificación promedio, precio, distancia y disponibilidad próxima.
5. Al aplicar filtros, el listado y el mapa se actualizan sin recargar toda la página.

Estado: ✅ Hecha

HU011 · Chat y confirmación de detalles del servicio

Yo como cliente, quiero comunicarme con el profesional dentro de la plataforma antes del servicio, para aclarar detalles del trabajo, materiales y condiciones.

Criterios de aceptación:

1. El sistema habilita el chat solo cuando exista una solicitud enviada o una reserva confirmada.
2. El chat guarda el historial de mensajes por solicitud o reserva.
3. El cliente puede enviar fotos o descripciones del problema (con validación de tamaño y formato).
4. El profesional puede confirmar por el chat detalles como materiales requeridos y tiempo estimado.
5. El sistema envía notificaciones cuando llega un mensaje nuevo.

Estado: ✅ Hecha

HU012 · Cancelación y reprogramación de reserva

Yo como cliente o profesional, quiero cancelar o reprogramar una reserva, para gestionar cambios de disponibilidad o imprevistos.

Criterios de aceptación:

1. El sistema permite cancelar antes de la hora programada con un motivo obligatorio.
2. Al cancelar, el sistema libera automáticamente el bloque de agenda correspondiente.
3. Para reprogramar, el usuario selecciona un nuevo horario disponible del profesional.
4. El estado de la reserva cambia a "Cancelado" o "Reprogramado" y queda registro del motivo y la fecha anterior.
5. Ambas partes reciben notificación de la cancelación o reprogramación.

Estado: ✅ Hecha

HU013 · Validación de identidad y antecedentes

Yo como usuario (cliente o profesional), quiero completar un proceso de verificación de identidad y antecedentes, para aumentar la seguridad y confianza en la plataforma.

Criterios de aceptación:

1. El sistema solicita documento de identidad y selfie para validación.
2. El sistema valida coincidencia entre selfie y foto del documento mediante el proveedor de verificación definido.
3. El sistema registra el resultado de la verificación (Aprobado/Rechazado/Pendiente) y la fecha.
4. Si la verificación es rechazada, el usuario recibe el motivo y puede reenviar la documentación.
5. Mientras el usuario esté "Pendiente" o "Rechazado", el sistema limita acciones críticas (publicar servicios, confirmar reservas) según el rol.

Estado: ✅ Hecha

HU014 · Revisión y aprobación de documentos del profesional

Yo como gestor de la aplicación (administrador/validador), quiero revisar las selfies y documentos cargados por el profesional, para aprobar o rechazar la verificación y asegurar la autenticidad de la información.

Criterios de aceptación:

1. El sistema muestra una bandeja de solicitudes "Pendiente de verificación" con filtros por fecha, tipo de documento y estado.
2. Al abrir una solicitud, el gestor visualiza: selfie, documento de identidad, certificados y evidencias; con opción de zoom/descarga.
3. El gestor puede marcar cada documento como "Aprobado" o "Rechazado" y agregar observaciones obligatorias cuando rechaza.
4. El gestor puede aprobar la solicitud solo si los documentos mínimos requeridos están completos y aprobados.
5. Al aprobar, el estado del profesional cambia a "Verificado" y se habilita la publicación/aceptación de servicios.
6. Al rechazar, el sistema notifica al profesional el motivo y mantiene el estado en "Rechazado" hasta que reenvíe la información.
7. El sistema registra auditoría de la revisión: quién revisó, fecha/hora y decisión tomada.

Estado: ✅ Hecha

HU016 · Historial de servicios del cliente

Yo como cliente, quiero ver mi historial de servicios solicitados, para llevar control de lo que he contratado.

Criterios de aceptación:

1. El sistema muestra lista de servicios realizados y pendientes.
2. Se visualiza fecha, profesional y estado.
3. Se puede acceder al detalle de cada servicio.
4. Los servicios cancelados también aparecen en el historial.

Estado: ✅ Hecha

HU017 · Historial de servicios del profesional

Yo como profesional, quiero ver mi historial de trabajos realizados, para llevar control de mis servicios.

Criterios de aceptación:

1. Se muestran todos los servicios realizados y cancelados.
2. Se visualiza fecha, cliente y estado.
3. Se puede ver el detalle de cada servicio.
4. Se muestra el total de servicios realizados.

Estado: ✅ Hecha

HU018 · Notificaciones del sistema

Yo como usuario, quiero recibir notificaciones, para estar informado sobre solicitudes y cambios.

Criterios de aceptación:

1. El sistema envía notificación cuando hay una nueva solicitud.
2. Notifica cuando un servicio es aceptado o rechazado.
3. Notifica mensajes nuevos en el chat.
4. Las notificaciones se pueden visualizar dentro del sistema.

Estado: ✅ Hecha

HU019 · Cambio de contraseña

Yo como usuario, quiero cambiar mi contraseña, para mantener mi cuenta segura.

Criterios de aceptación:

1. Debo ingresar mi contraseña actual.
2. La nueva contraseña debe cumplir requisitos mínimos.
3. El sistema confirma el cambio exitoso.
4. Si la contraseña actual es incorrecta, muestra error.

Estado: ✅ Hecha

HU020 · Recuperación de contraseña

Yo como usuario, quiero recuperar mi contraseña, para poder acceder si la olvido.

Criterios de aceptación:

1. El usuario ingresa su correo.
2. El sistema envía un enlace de recuperación.
3. El enlace permite crear una nueva contraseña.
4. El enlace tiene un tiempo de expiración.

Estado: ✅ Hecha

HU021 · Cerrar sesión

Yo como usuario, quiero cerrar sesión, para proteger mi cuenta cuando termine de usarla.

Criterios de aceptación:

1. Existe un botón visible para cerrar sesión.
2. Al cerrar sesión se redirige al inicio.
3. Se elimina la sesión activa.

Estado: ✅ Hecha

HU022 · Registro de ubicación manual

Yo como usuario, quiero ingresar mi ubicación manualmente, para usar la plataforma si no quiero activar GPS.

Criterios de aceptación:

1. El usuario puede escribir su dirección o comuna.
2. El sistema valida la ubicación ingresada.
3. Se usa esa ubicación para búsquedas.
4. Se puede actualizar en cualquier momento.

Estado: ✅ Hecha

HU023 · Eliminación de cuenta

Yo como usuario, quiero eliminar mi cuenta, para dejar de usar la plataforma.

Criterios de aceptación:

1. El sistema solicita confirmación antes de eliminar.
2. Se valida que no tenga servicios pendientes.
3. La cuenta se elimina del sistema.
4. Se muestra mensaje de confirmación.

Estado: ✅ Hecha

HU024 · Reporte de usuarios

Yo como usuario, quiero reportar a otro usuario, para informar comportamientos indebidos.

Criterios de aceptación:

1. Se puede reportar desde el perfil o servicio.
2. Se debe ingresar un motivo.
3. El reporte queda registrado en el sistema.
4. El administrador puede revisar los reportes.

Estado: ✅ Hecha

HU025 · Visualización de disponibilidad del profesional

Yo como cliente, quiero ver los horarios disponibles del profesional, para elegir el mejor momento para mi servicio.

Criterios de aceptación:

1. Se muestra un calendario con horarios disponibles.
2. No se muestran horarios ocupados.
3. Se actualiza en tiempo real.
4. Permite seleccionar una fecha y hora fácilmente.

Estado: ✅ Hecha

HU026 · Estado del servicio

Yo como usuario, quiero ver el estado del servicio, para saber en qué etapa se encuentra.

Criterios de aceptación:

1. El servicio puede estar en estados como pendiente, confirmado, en proceso o completado.
2. El estado se actualiza automáticamente.
3. El usuario puede ver el estado en su historial.
4. Se notifica cuando cambia el estado.

Estado: ✅ Hecha

HU027 · Confirmación de inicio de servicio

Yo como profesional, quiero marcar el inicio del servicio, para indicar que ya comencé el trabajo.

Criterios de aceptación:

1. El profesional puede iniciar el servicio desde la reserva.
2. El estado cambia a "En proceso".
3. El cliente recibe notificación.
4. Solo se puede iniciar en la hora programada o después.

Estado: ✅ Hecha

HU028 · Confirmación de finalización de servicio

Yo como profesional, quiero marcar el servicio como finalizado, para cerrar el trabajo realizado.

Criterios de aceptación:

1. El profesional puede finalizar el servicio.
2. El estado cambia a "Completado".

3. Se habilita la opción de calificación.
4. El cliente recibe notificación.

Estado: ✅ Hecha

HU029 · Visualización de comentarios

Yo como usuario, quiero ver los comentarios de otros usuarios, para tomar mejores decisiones.

Criterios de aceptación:

1. Se muestran comentarios en el perfil.
2. Cada comentario incluye calificación y texto.
3. Se ordenan por fecha o relevancia.
4. Solo usuarios que usaron el servicio pueden comentar.

Estado: ✅ Hecha

HU030 · Filtro por disponibilidad inmediata

Yo como cliente, quiero filtrar profesionales disponibles de inmediato, para encontrar ayuda rápida.

Criterios de aceptación:

1. Existe un filtro de “disponible ahora”.
2. Solo muestra profesionales con horario libre cercano.
3. Se actualiza en tiempo real.
4. Se combina con otros filtros (precio, calificación, etc.).

Estado: ✅ Hecha

HU031 · Edición de perfil del cliente

Yo como cliente, quiero editar mi perfil, para mantener mis datos actualizados.

Criterios de aceptación:

1. Puedo cambiar nombre, foto y datos básicos.
2. El sistema guarda los cambios correctamente.
3. Se validan los campos obligatorios.
4. Los cambios se reflejan inmediatamente.

Estado: ✅ Hecha

HU032 · Subida de foto de perfil

Yo como usuario, quiero subir una foto de perfil, para que otros me identifiquen fácilmente.

Criterios de aceptación:

1. Se permite subir imágenes.
2. El sistema valida formato y tamaño.
3. La foto se actualiza en el perfil.
4. Se puede cambiar en cualquier momento.

Estado: ✅ Hecha

HU033 · Visualización de servicios por categoría

Yo como cliente, quiero ver servicios por categoría, para encontrar más fácil lo que necesito.

Criterios de aceptación:

1. Se muestran categorías disponibles.
2. Al seleccionar una, se listan los profesionales.
3. Se puede navegar entre categorías.
4. Se muestran resultados relacionados.

Estado: ✅ Hecha

HU034 · Guardar profesionales favoritos

Yo como cliente, quiero guardar profesionales como favoritos, para encontrarlos rápidamente después.

Criterios de aceptación:

1. Existe opción de agregar a favoritos.
2. Se pueden ver en una lista de favoritos.
3. Se pueden eliminar de favoritos.
4. Se mantienen guardados en la cuenta.

Estado: ✅ Hecha

HU035 · Visualización de favoritos

Yo como cliente, quiero ver mis profesionales favoritos, para acceder rápido a ellos.

Criterios de aceptación:

1. Se muestra lista de favoritos.
2. Se puede acceder al perfil de cada uno.
3. Se puede eliminar de la lista.
4. La lista se guarda correctamente.

Estado: ✅ Hecha

HU036 · Confirmación de reserva por el cliente

Yo como cliente, quiero confirmar una reserva, para asegurar que el servicio se realizará.

Criterios de aceptación:

1. El cliente puede confirmar después de aceptación.
2. El sistema muestra el estado confirmado.
3. Se notifica al profesional.
4. Se guarda el registro.

Estado: ✅ Hecha

HU037 · Visualización de ingresos del profesional

Yo como profesional, quiero ver mis ingresos generados, para llevar control de lo que gano.

Criterios de aceptación:

1. Se muestra total de ingresos.
2. Se puede ver por fechas.
3. Se listan servicios asociados.
4. Los datos se actualizan automáticamente.

Estado: ✅ Hecha

HU038 · Bloqueo de usuarios problemáticos

Yo como administrador, quiero bloquear usuarios, para evitar comportamientos indebidos.

Criterios de aceptación:

1. El administrador puede bloquear usuarios.
2. El usuario bloqueado no puede acceder.
3. Se guarda el motivo del bloqueo.
4. Se puede desbloquear posteriormente.

Estado: ✅ Hecha

HU039 · Búsqueda por nombre

Yo como cliente, quiero buscar profesionales por nombre, para encontrarlos directamente.

Criterios de aceptación:

1. Existe barra de búsqueda.
2. Se muestran resultados coincidentes.
3. Se puede seleccionar el perfil.
4. Funciona en tiempo real.

Estado: ✅ Hecha

HU040 · Visualización de distancia

Yo como cliente, quiero ver la distancia del profesional, para saber qué tan cerca está.

Criterios de aceptación:

1. Se muestra distancia en km.
2. Se actualiza según ubicación.
3. Aparece en listados y perfil.
4. Se usa para ordenar resultados.

Estado: ✅ Hecha

HU041 · Indicador de estado en línea

Yo como usuario, quiero ver si el otro usuario está en línea, para saber si puedo comunicarme rápido.

Criterios de aceptación:

1. Se muestra indicador de en línea o desconectado.
2. Se actualiza automáticamente.
3. Aparece en el chat.
4. Es visible para ambas partes.

Estado: ✅ Hecha

HU042 · Adjuntar archivos en el chat

Yo como usuario, quiero enviar archivos en el chat, para compartir información del servicio.

Criterios de aceptación:

1. Se permite enviar imágenes.
2. Se valida tamaño y formato.
3. El archivo se visualiza en el chat.
4. Se guarda en el historial.

Estado: ✅ Hecha

HU043 · Eliminación de mensajes

Yo como usuario, quiero eliminar mensajes enviados, para corregir errores.

Criterios de aceptación:

1. Se puede eliminar un mensaje.
2. El mensaje desaparece del chat.
3. Se actualiza para ambas partes.
4. Se mantiene la conversación.

Estado: ✅ Hecha

HU044 · Visualización de perfil del cliente

Yo como profesional, quiero ver el perfil del cliente, para conocer con quién trabajaré.

Criterios de aceptación:

1. Se muestra nombre y foto.
2. Se muestra calificación del cliente.

3. Se pueden ver comentarios.
4. Se accede desde la solicitud.

Estado: ✅ Hecha

HU045 · Alerta por baja calificación

Yo como administrador, quiero recibir alertas de usuarios con baja calificación, para revisar posibles problemas.

Criterios de aceptación:

1. El sistema detecta calificación baja.
2. Se genera una alerta automática.
3. Se puede revisar el perfil.
4. Se guarda el registro.

Estado: ✅ Hecha

HU046 · Ordenamiento por precio

Yo como cliente, quiero ordenar por precio, para elegir según mi presupuesto.

Criterios de aceptación:

1. Se puede ordenar de menor a mayor.
2. También de mayor a menor.
3. Se actualiza el listado.
4. Funciona junto con otros filtros.

Estado: ✅ Hecha

HU047 · Visualización de tiempo estimado

Yo como cliente, quiero ver el tiempo estimado del servicio, para organizar mi tiempo.

Criterios de aceptación:

1. El profesional indica duración estimada.
2. Se muestra en la reserva.
3. El cliente puede visualizarlo antes de confirmar.
4. Se guarda con el servicio.

Estado: ✅ Hecha

HU048 · Validación de datos obligatorios

Yo como usuario, quiero que el sistema valide datos obligatorios, para evitar errores al registrar información.

Criterios de aceptación:

1. El sistema detecta campos vacíos.
2. Muestra mensajes de error.
3. No permite continuar si faltan datos.
4. Indica claramente qué falta.

Estado: ✅ Hecha

HU049 · Visualización de servicios en proceso

Yo como usuario, quiero ver mis servicios en proceso, para saber cuáles están activos.

Criterios de aceptación:

1. Se muestran servicios en curso.
2. Se visualiza estado actual.
3. Se puede acceder al detalle.
4. Se actualiza en tiempo real.

Estado: ✅ Hecha

HU050 · Cierre de sesión por inactividad

Yo como usuario, quiero que el sistema cierre mi sesión por inactividad, para proteger mi cuenta.

Criterios de aceptación:

1. El sistema detecta inactividad.
2. Cierra sesión automáticamente.
3. Muestra mensaje informativo.
4. Redirige al inicio de sesión.

Estado: ✅ Hecha

3. Requisitos no funcionales

Se formulan con el modelo de calidad **ISO/IEC 25010**. Cada uno tiene una meta medible, la forma en que el sistema la cumple y cómo se verifica. El estado indica si ya está cumplido, si es una meta que se mide en el despliegue o si tiene una acción pendiente.

3.1 Seguridad

ID	Requisito	Cómo se cumple	Verificación	Estado
RNF-SEG-01	Solo usuarios autenticados acceden a sus datos	Autenticación por token (Laravel Sanctum) en todas las rutas privadas	Tests de 401 sin token en cada módulo	✅
RNF-SEG-02	Cada usuario solo opera sobre sus recursos y según su rol	Middleware `role:` (tipo de cuenta) y Policies (dueño del recurso); los usuarios bloqueados no entran (`not_blocked`)	Tests de 403 por endpoint (otro usuario, otro rol)	✅
RNF-SEG-03	Contraseñas protegidas	Hash bcrypt (cast `hashed`), mínimo 8 caracteres y confirmación	Tests de registro	✅
RNF-SEG-04	Resistencia a fuerza bruta en el acceso	Máximo 10 intentos por minuto en registro, inicio de sesión y recuperación de contraseña (`throttle:10,1`)	Revisión de rutas; prueba manual	✅
RNF-SEG-05	Toda entrada se valida en el servidor	FormRequest por endpoint con errores 422 por campo. El cliente valida además con Zod, sin reemplazar al servidor	Tests de validación en cada endpoint	✅
RNF-SEG-06	Los documentos de identidad y los adjuntos del chat no son públicos	Disco privado y URL firmada que vence (60 min); sin firma responde 403	ChatTest; prueba manual de URL sin firma	✅
RNF-SEG-07	Tráfico cifrado	HTTPS (TLS) y WebSockets seguros (`wss`) en producción	Checklist de despliegue	🎯 En despliegue
RNF-SEG-08	La API solo acepta peticiones del frontend oficial	Publicar `config/cors.php` con el dominio del frontend en producción (hoy acepta cualquier origen, valor por defecto de Laravel)	Checklist de despliegue	⚠️ Pendiente
RNF-SEG-09	Las dependencias no tienen vulnerabilidades conocidas	Auditoría de dependencias (`composer audit`, `npm audit`) en cada PR	CI (job "Seguridad de dependencias")	✅
RNF-SEG-10	No se puede averiguar qué correos tienen cuenta	La recuperación de contraseña responde igual exista o no el correo	PasswordResetTest	✅
RNF-SEG-11	Las sesiones abandonadas se cierran	Cierre automático tras 15 minutos sin actividad en el navegador (HU050)	Prueba manual	✅

3.2 Privacidad y protección de datos

ID	Requisito	Cómo se cumple	Verificación	Estado
RNF-PRI-01	El perfil público no expone datos de contacto	`ProfessionalResource` excluye correo, teléfono, dirección y coordenadas	ProfessionalsTest	✅
RNF-PRI-02	El chat solo lo ven las partes de la reserva	Policy `chat` (ni otros usuarios ni el administrador)	ChatTest	✅
RNF-PRI-05	El perfil de un cliente solo lo ve el profesional que tiene una reserva con él	Policy `viewClientProfile` (HU044)	UserReportsAndModerationTest	✅

ID	Requisito	Cómo se cumple	Verificación	Estado
RNF-PRI-03	Autorización informada para el tratamiento de datos (Ley 1581)	Aceptación explícita de la política de tratamiento al registrarse	—	⚠️ Pendiente (ver 07)
RNF-PRI-04	Derecho de supresión	Eliminación de cuenta con anonimización: se borran datos de contacto, ubicación, fotos y documentos, y se conserva el historial de la otra parte (HU023, decisión D12)	AccountTest	✅

3.3 Integridad y auditoría

ID	Requisito	Cómo se cumple	Verificación	Estado
RNF-INT-01	Cada cambio de estado de una reserva queda registrado con autor, fecha y motivo	Tabla `booking_status_changes`; todo cambio pasa por `BookingService::transition()`	BookingHistoryTest, BookingsTest	✅
RNF-INT-02	No se pueden reservar dos servicios a la misma hora	Validación de cruces con descanso y bloqueo de la fila del profesional dentro de la transacción (`lockForUpdate`)	BookingsTest, ProfessionalSlotsTest	✅
RNF-INT-03	Los estados solo cambian por transiciones válidas	Máquina de estados en `BookingStatus`; una transición inválida responde 409	Tests de 409 en cada acción	✅
RNF-INT-04	Integridad referencial	Llaves foráneas en todas las relaciones; los servicios con reservas no se borran (se ocultan)	Migraciones; ProfessionalProfileTest	✅
RNF-INT-05	El precio pactado no cambia después de reservar	Copia del precio en `bookings.agreed_price`	BookingsTest, ProfessionalProfileTest	✅

3.4 Eficiencia de desempeño

ID	Requisito	Cómo se cumple	Verificación	Estado
RNF-REN-01	El 95 % de las respuestas de la API en menos de 500 ms con la carga esperada del piloto	Listados paginados (15–20, máximo 50), índices en las columnas de búsqueda, carga anticipada de relaciones (`with`), promedio de calificación precalculado (D13)	Medición con el servidor de producción	🎯 En despliegue
RNF-REN-02	Eventos en tiempo real entregados en menos de 2 s	Laravel Reverb con cola; los eventos se emiten después de confirmar la transacción	Prueba manual con dos sesiones	✅ (local)
RNF-REN-03	Carga inicial del frontend ligera	Rutas cargadas bajo demanda (*lazy loading*) y build optimizado de Vite	Build en CI	✅

3.5 Disponibilidad y fiabilidad

ID	Requisito	Cómo se cumple	Verificación	Estado
RNF-DIS-01	Disponibilidad mensual ≥ 99 % durante el piloto	Procesos supervisados (Supervisor reinicia la cola y Reverb) y verificación de salud (`/up`)	Monitoreo externo	🎯 En despliegue
RNF-DIS-02	Recuperación ante pérdida de datos	Respaldo diario de la base y los archivos, con 7 días de retención; restauración documentada	Prueba de restauración (06)	🎯 En despliegue
RNF-DIS-03	Un fallo del tiempo real no impide usar la app	Sin WebSocket la app sigue funcionando y se actualiza al recargar o al volver a la pestaña	Prueba manual con Reverb apagado	✅

3.6 Usabilidad y accesibilidad

ID	Requisito	Cómo se cumple	Verificación	Estado
RNF-USA-01	Interfaz usable en celular, tableta y computador	Diseño responsive con Tailwind; menú móvil	Revisión manual en tres anchos	✅
RNF-USA-02	Mensajes claros y en español	Mensajes de validación y de error en español, con el nombre del campo	ApiErrorsTest y tests de cada formulario	✅
RNF-USA-03	Toda vista con datos muestra cargando, error, vacío y contenido	Patrón obligatorio del frontend (`docs/arquitectura.md`)	Revisión en PR; tests de componentes	✅

ID	Requisito	Cómo se cumple	Verificación	Estado
RNF-USA-04	Accesibilidad base (meta: WCAG 2.1 nivel AA)	HTML semántico, `aria-label` en controles de solo ícono, foco visible, contraste de la paleta, diálogos que se cierran con Esc	Revisión manual y auditoría Lighthouse	🎯 Auditoría pendiente
RNF-USA-05	Identidad visual consistente	Sistema de diseño con tokens; un verificador en CI rechaza colores fuera de la paleta	`npm run lint:tokens`	✅

3.7 Mantenibilidad

ID	Requisito	Cómo se cumple	Verificación	Estado
RNF-MAN-01	Código modular y por capas	Backend: Ruta → FormRequest → Policy → Service → Resource. Frontend: módulos autocontenidos con API pública	Revisión en PR; regla de lint de imports	✅
RNF-MAN-02	Cada cambio pasa pruebas automáticas antes de unirse	CI obligatorio en `main`: tests, formato, lint, tipos y build	Protección de la rama	✅
RNF-MAN-03	Reglas de calidad protegidas	Check **Guardas**: la configuración de calidad no se modifica sin aprobación explícita	CI	✅
RNF-MAN-04	Contrato de API documentado y sincronizado	`docs/api/endpoints.md` se actualiza en el mismo PR; especificación OpenAPI generada del código	Revisión en PR	✅

3.8 Compatibilidad y portabilidad

ID	Requisito	Cómo se cumple	Verificación	Estado
RNF-COM-01	Navegadores modernos	Últimas dos versiones de Chrome, Edge, Firefox y Safari	Revisión manual	✅
RNF-COM-02	Versiones de PHP soportadas	PHP 8.2 a 8.5	CI corre las pruebas en 8.2 y 8.5	✅
RNF-COM-03	Base de datos	MySQL 8 en desarrollo y producción; SQLite en memoria para las pruebas	CI	✅

3.9 Escalabilidad

ID	Requisito	Cómo se cumple	Estado
RNF-ESC-01	La API puede crecer horizontalmente	API sin estado de sesión (tokens); colas para el trabajo diferido	✅ Diseño
RNF-ESC-02	El tiempo real puede crecer	Reverb admite varias instancias coordinadas con Redis	🎯 Futuro
RNF-ESC-03	Los archivos pueden salir del servidor	Discos de Laravel intercambiables (local → S3) sin cambiar el código	🎯 Futuro

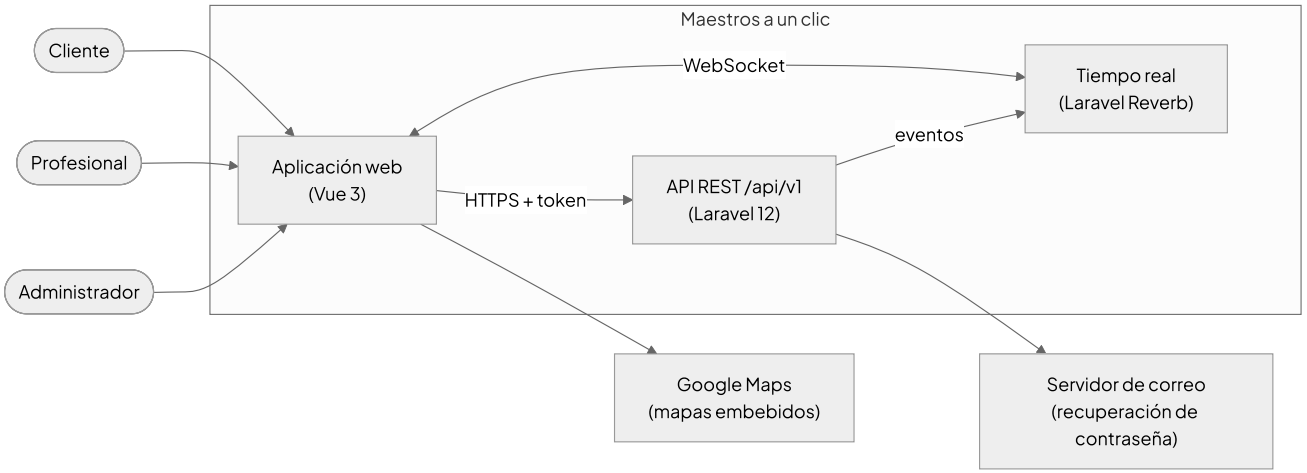
Leyenda: ✅ cumplido · 🎯 meta que se verifica en el despliegue · ⚠️ acción pendiente.

Diagramas del sistema

Diagramas de análisis y diseño hechos a partir del sistema actual: modelos, migraciones, rutas y servicios. Están en Mermaid para que se versionen con el código: GitHub los dibuja al abrir este archivo y el generador de PDF los incluye. Reemplazan a los diagramas PNG del entregable original, que eran anteriores al modelo definitivo.

1. Contexto del sistema

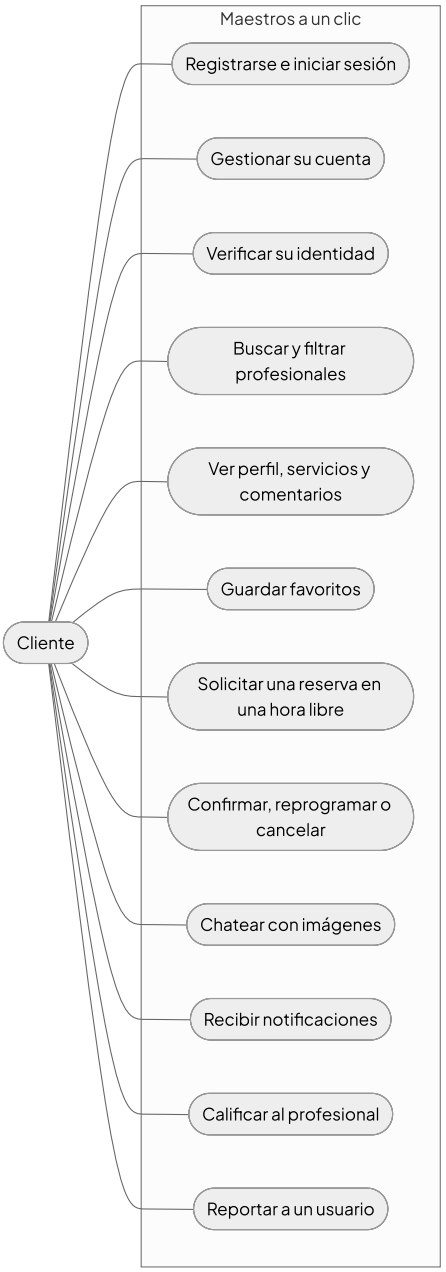
Quién usa el sistema y con qué se integra.



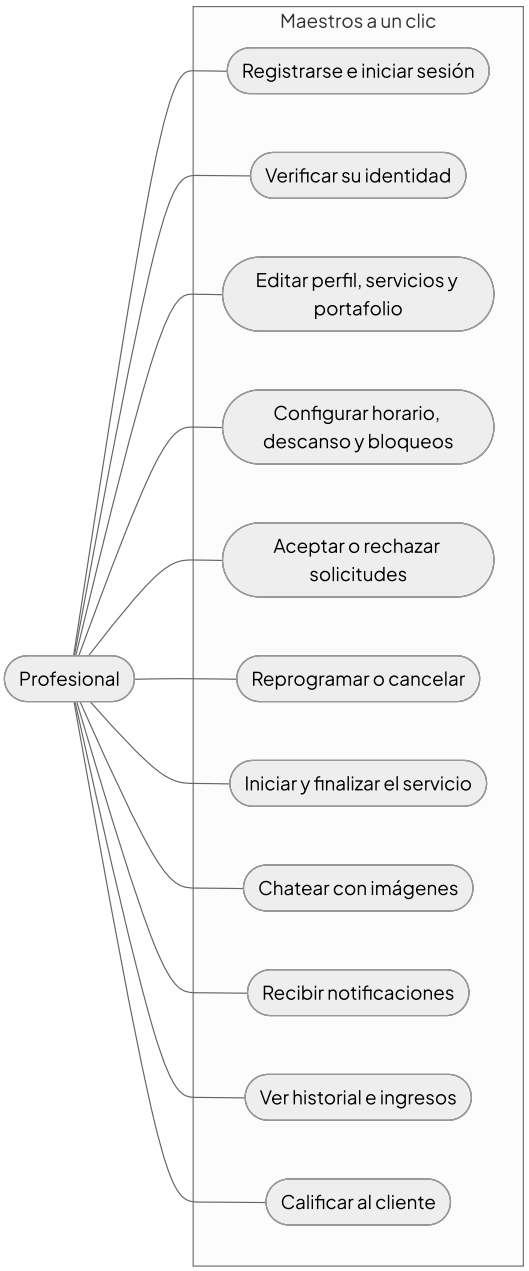
2. Casos de uso

Un diagrama por actor. Las historias que cubre cada caso están en 02-requisitos.md.

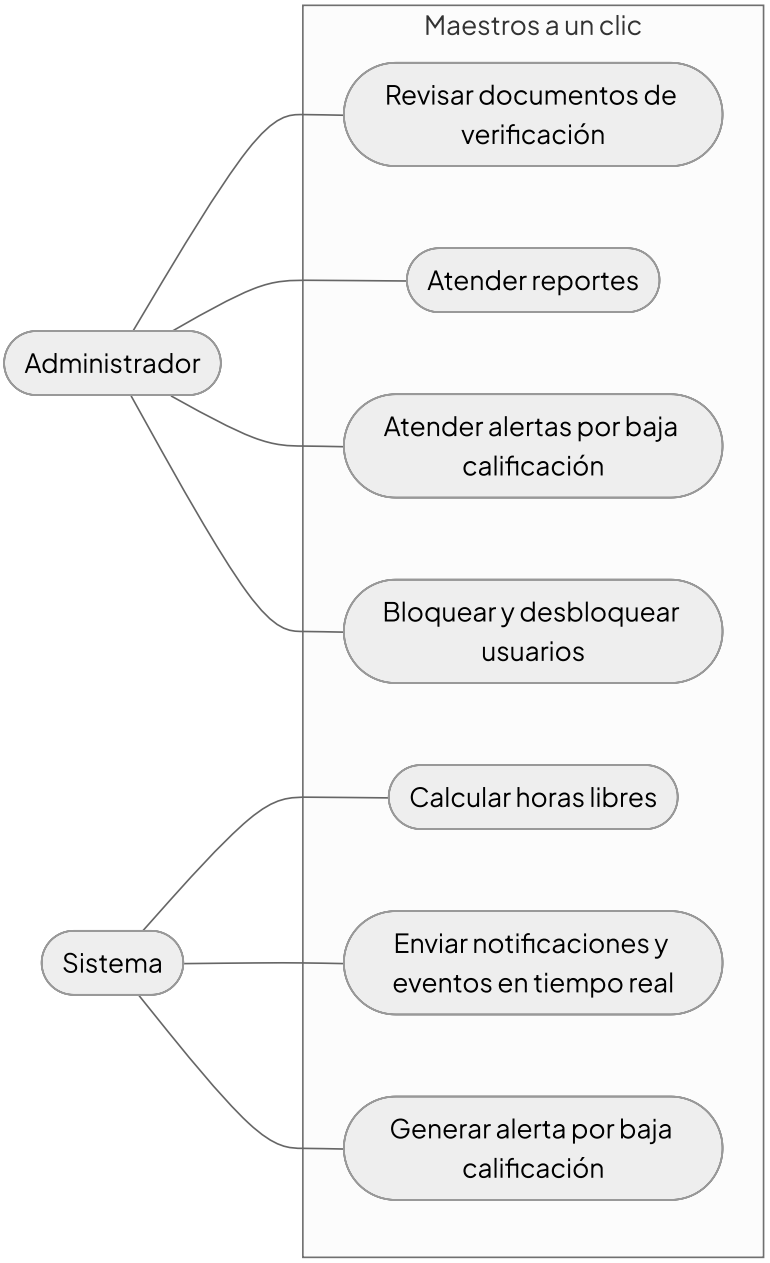
2.1 Cliente


Maestros a un clic
Registrarse e iniciar sesión
Gestionar su cuenta
Verificar su identidad
Buscar y filtrar profesionales
Ver perfil, servicios y comentarios
Guardar favoritos
Cliente
Solicitar una reserva en una hora libre
Confirmar, reprogramar o cancelar
Chatear con imágenes
Recibir notificaciones
Calificar al profesional
Reportar a un usuario

2.2 Profesional

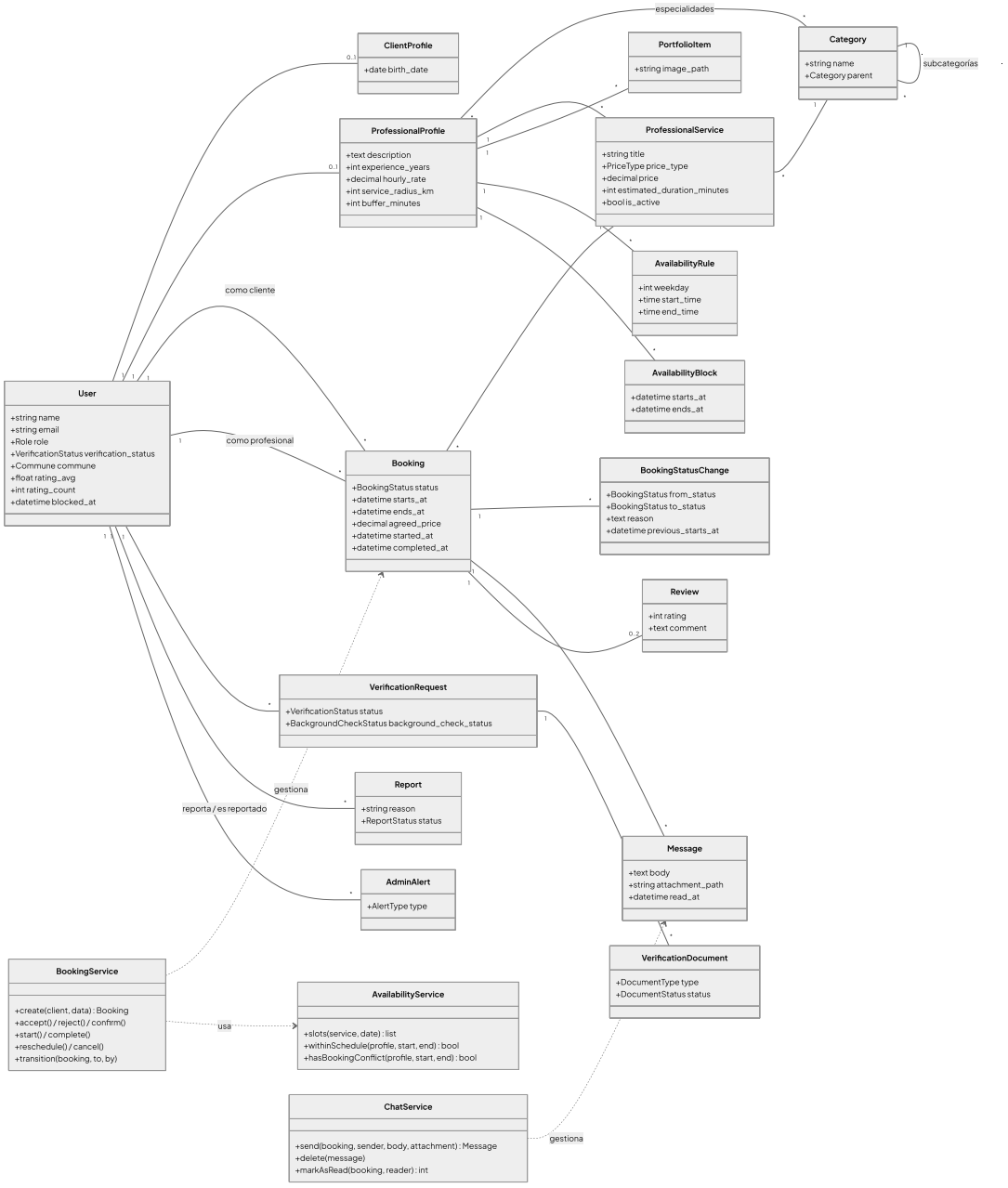


2.3 Administrador y sistema



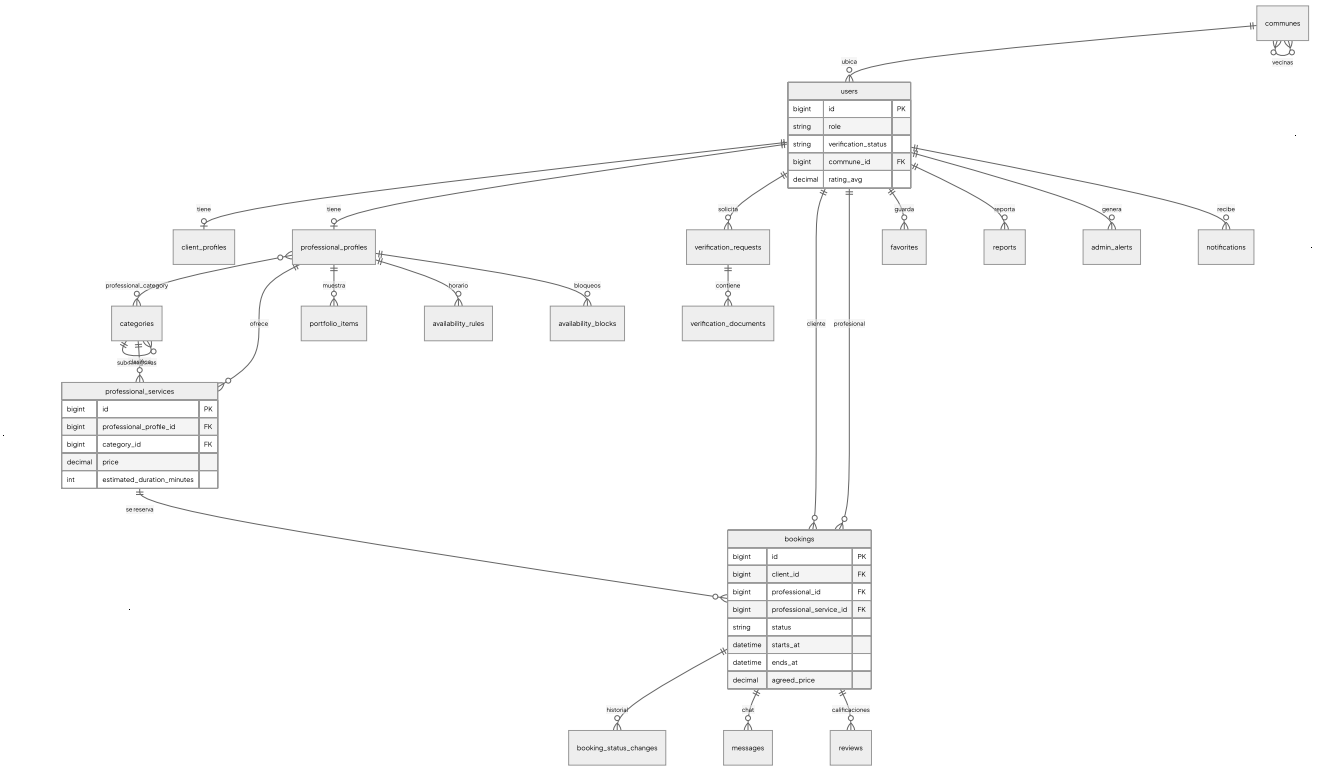
3. Clases del dominio

Modelos principales con sus atributos clave y los servicios que aplican las reglas de negocio. Los estados, roles y tipos son enumeraciones de PHP (`app/Enums`).



4. Modelo entidad-relación

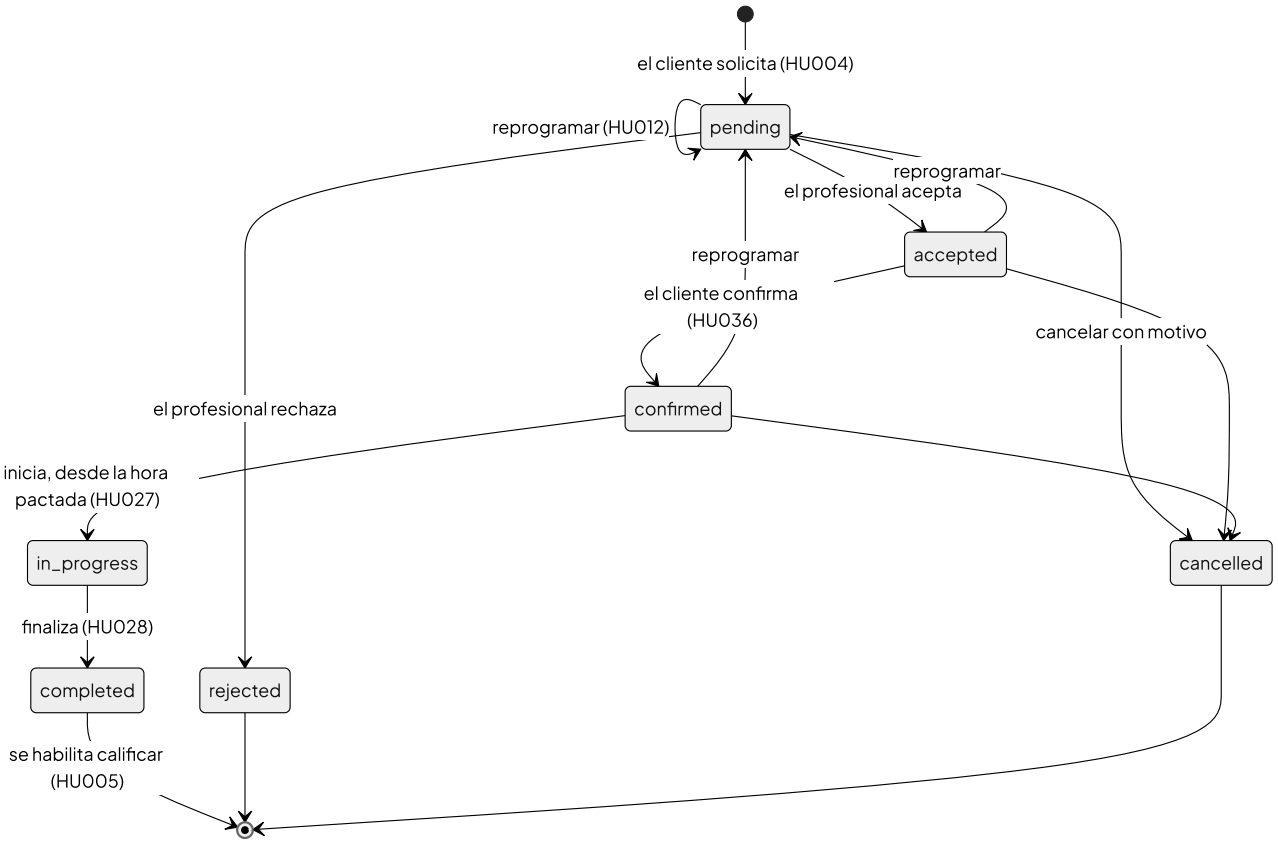
Entidades y relaciones con sus llaves. Las columnas completas de cada tabla están en docs/modelo-de-datos.md §2.



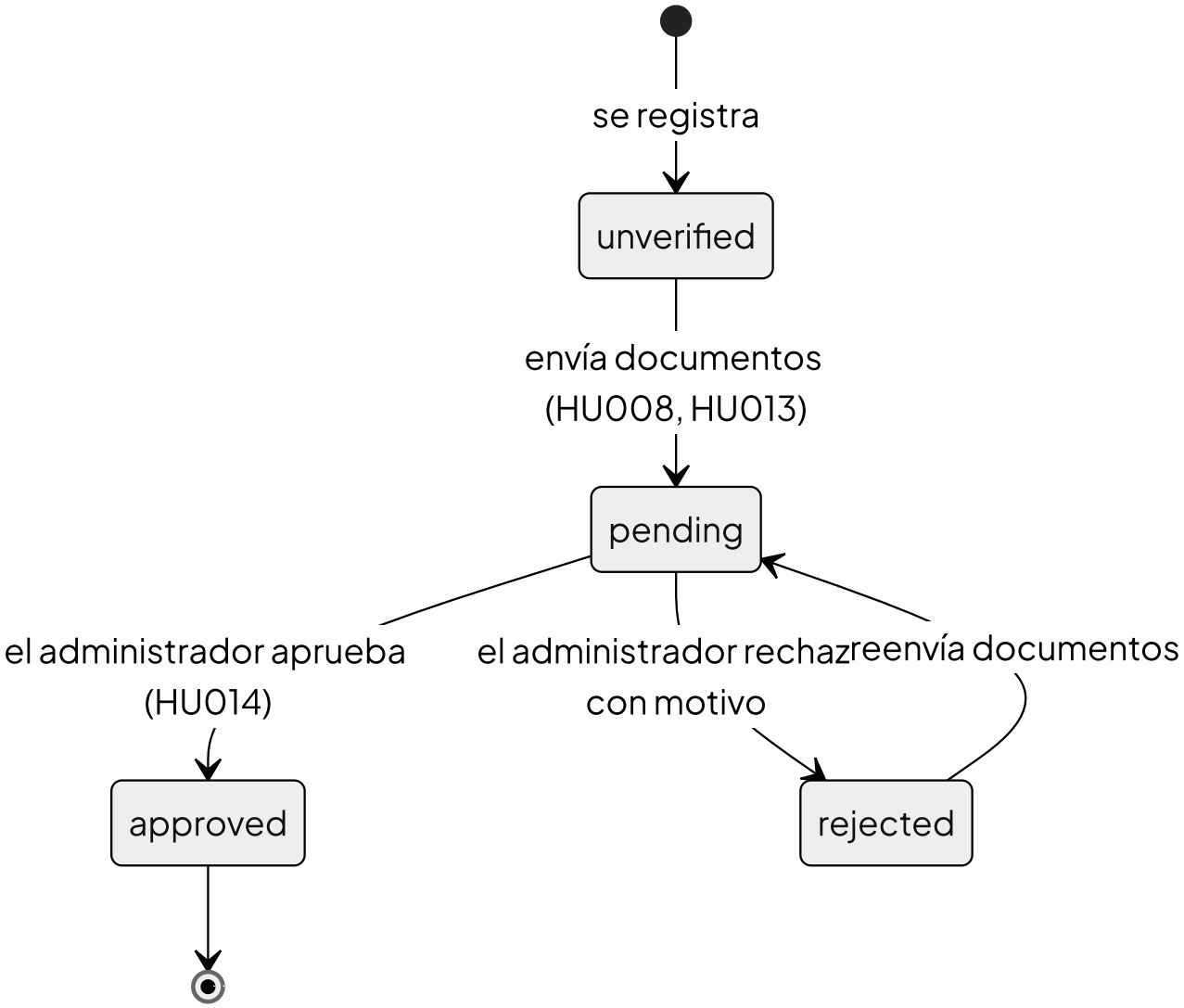
5. Estados

5.1 Reserva

Todo cambio pasa por `BookingService::transition()` y queda en `booking_status_changes`.

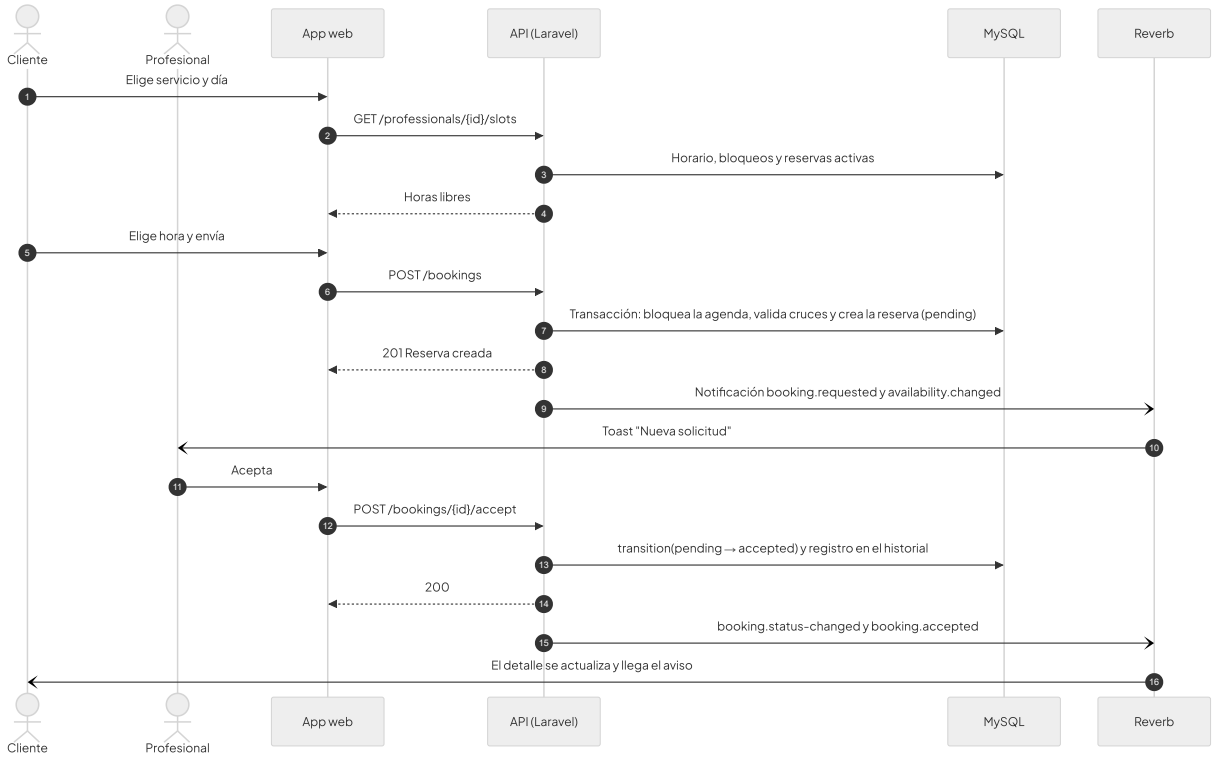


5.2 Verificación de identidad

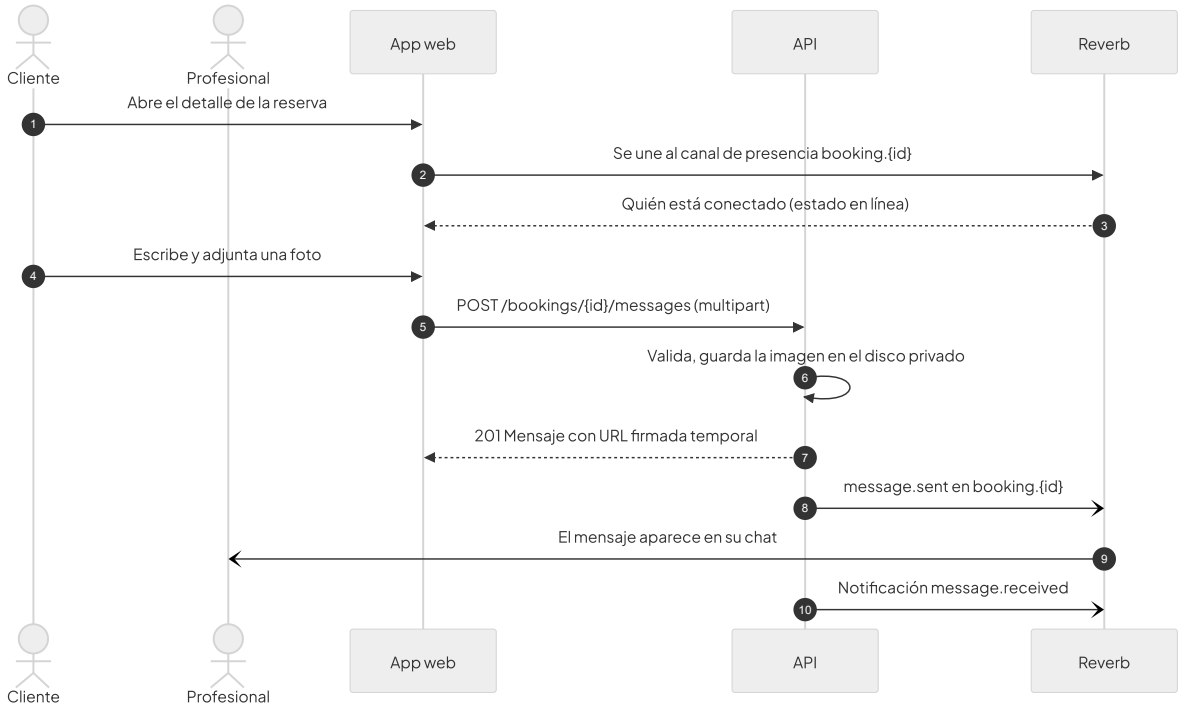


6. Secuencias

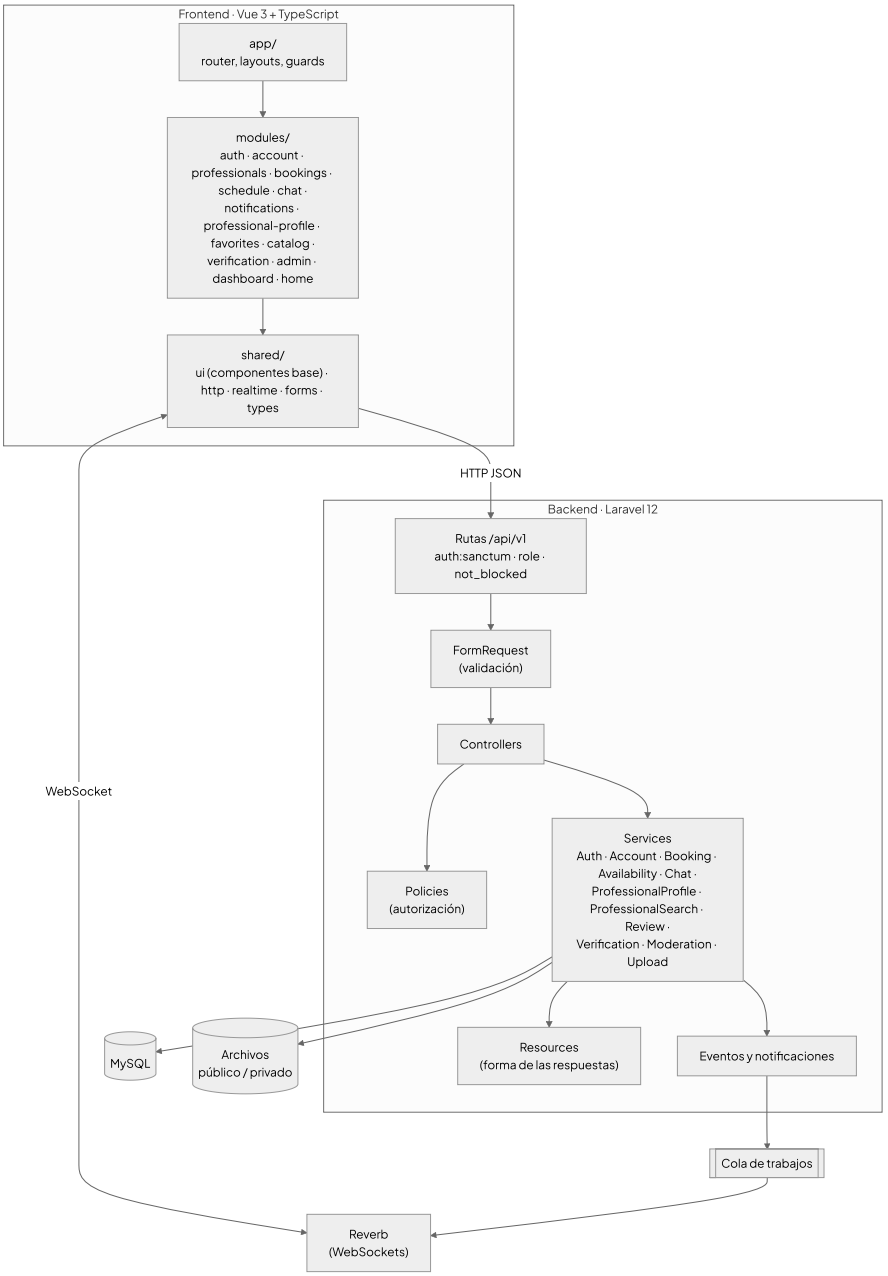
6.1 Solicitar y aceptar una reserva



6.2 Mensaje en el chat

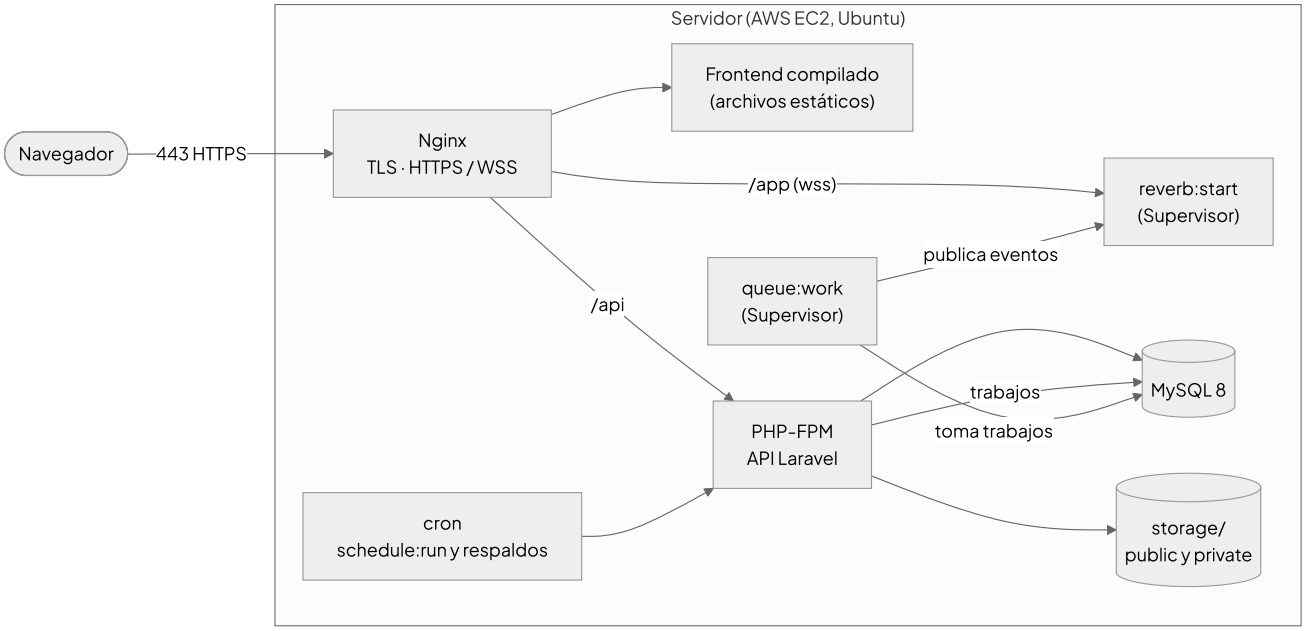


7. Componentes



8. Despliegue

Topología prevista para producción. Los pasos están en 06-despliegue-y-operacion.md.


Servidor (AWS EC2, Ubuntu)
Navegador
443 HTTPS
Nginx
TLS · HTTPS / WSS
Frontend compilado
(archivos estáticos)
/app (wss)
reverb:start
(Supervisor)
publica eventos
queue:work
(Supervisor)
/api
trabajos
MySQL 8
PHP-FPM
API Laravel
toma trabajos
cron
schedule:run y respaldos
storage/
public y private

Matriz de trazabilidad

Relaciona cada historia de usuario con lo que la implementa (endpoints y eventos del backend, vistas y componentes del frontend) y con las pruebas automatizadas que la verifican. Sirve para comprobar que cada requisito está cubierto y para ubicar rápido el código de una funcionalidad.

- **Backend:** rutas bajo `/api/v1` (contrato) y eventos de tiempo real (tiempo-real.md).
- **Frontend:** vistas en `src/modules/<módulo>/views` y componentes en `src/modules/<módulo>/components` del repositorio del frontend.
- **Pruebas:** `*Test` son pruebas de API del backend (`tests/Feature`); `*.spec` son pruebas del frontend (`__tests__` de cada módulo).
- **Estado:** el de su issue en GitHub al generar el documento.

HU	Módulo	Backend (API y eventos)	Frontend	Pruebas	Estado
HU001 Inicio de sesión	auth	POST /auth/login	LoginView (/login)	Auth/LoginTest · auth/schemas.spec	✅ Hecha
HU002 Búsqueda por geolocalización	búsqueda	GET /professionals?lat=&lng= (radio de acción, comunas aledañas)	SearchResultsView (geolocalización del navegador)	ProfessionalsTest	✅ Hecha
HU003 Gestión de disponibilidad y agenda	agenda	GET /professional/availability · PUT …/rules · POST/DELETE …/blocks · evento availability.changed	AvailabilityView (WeeklyScheduleEditor, BlocksManager)	AvailabilityTest · WeeklyScheduleEditor.spec · weekly.spec	✅ Hecha
HU004 Solicitud y aceptación de servicio	reservas	POST /bookings · POST /bookings/{id}/accept · /reject · GET /professionals/{id}/slots	NewBookingView · BookingResponseActions · PendingRequestsPanel	BookingsTest · ProfessionalSlotsTest · BookingResponseActions.spec · SlotPicker.spec	✅ Hecha
HU005 Sistema de calificación recíproca	calificaciones	POST /bookings/{id}/review	ReviewBookingModal (detalle de la reserva)	ReviewsTest · ReviewBookingModal.spec	✅ Hecha
HU006 Registro de cliente	auth	POST /auth/register/client	RegisterClientView	RegisterClientTest · auth/schemas.spec	✅ Hecha
HU007 Registro de profesional	auth	POST /auth/register/professional	RegisterProfessionalView	Auth/RegisterProfessionalTest · auth/schemas.spec	✅ Hecha
HU008 Cargue de documentos	verificación-admin	GET/POST /verification	VerificationView	VerificationTest · verification/schemas.spec	✅ Hecha
HU009 Edición de perfil profesional	perfil-profesional	GET/PATCH /professional/profile · PUT /professional/categories · /professional/services · /professional/portfolio	AboutMeView · MyServicesView	ProfessionalProfileTest · professional-profile/schemas.spec · ServiceFormModal.spec	✅ Hecha
HU010 Filtros y ordenamiento de profesionales	búsqueda	GET /professionals (categoría, precio, orden)	SearchResultsView	ProfessionalsTest	✅ Hecha
HU011 Chat y confirmación de detalles del servicio	chat-notificaciones	GET/POST /bookings/{id}/messages · POST …/messages/read · evento message.sent	BookingChat (detalle de la reserva)	ChatTest · BookingChat.spec	✅ Hecha
HU012 Cancelación y reprogramación de reserva	reservas	POST /bookings/{id}/reschedule · POST /bookings/{id}/cancel	RescheduleBookingModal · CancelBookingModal	BookingRescheduleTest · BookingsTest · RescheduleBookingModal.spec	✅ Hecha
HU013 Validación de identidad y antecedentes	verificación-admin	POST /verification/documents/{id} · PATCH /admin/verification-documents/{id}	VerificationView · ReplaceDocumentModal	VerificationTest · AdminVerificationTest · ReplaceDocumentModal.spec	✅ Hecha

HU	Módulo	Backend (API y eventos)	Frontend	Pruebas	Estado
HU014 Revisión y aprobación de documentos del profesional	verificación-admin	/admin/verifications/* (aprobar, rechazar, ver archivo)	AdminVerificationsView · AdminVerificationDetailModal	AdminVerificationTest · AdminVerifications.spec	✅ Hecha
HU016 Historial de servicios del cliente	reservas	GET /bookings	MyBookingsView (filtros por estado)	BookingsTest · BookingList.spec	✅ Hecha
HU017 Historial de servicios del profesional	reservas	GET /bookings · GET /bookings/summary	ProfessionalBookingsView (totales y filtros)	BookingsTest · BookingList.spec	✅ Hecha
HU018 Notificaciones del sistema	chat-notificaciones	GET /notifications · POST /notifications/{id}/read · /read-all · canal App.Models.User.{id}	NotificationBell (barra superior)	NotificationsTest · NotificationBell.spec	✅ Hecha
HU019 Cambio de contraseña	cuenta	PUT /me/password	SecuritySettingsView · ChangePasswordForm	AccountTest · ChangePasswordForm.spec	✅ Hecha
HU020 Recuperación de contraseña	cuenta	POST /auth/forgot-password · POST /auth/reset-password	ForgotPasswordView · ResetPasswordView	Auth/PasswordResetTest · PasswordRecovery.spec	✅ Hecha
HU021 Cerrar sesión	auth	POST /auth/logout	AppNavbar (Salir)	Auth/SessionTest · router.spec	✅ Hecha
HU022 Registro de ubicación manual	cuenta	PATCH /me (comuna, dirección, coordenadas)	LocationSettingsView · ManualLocationForm	AccountTest · ManualLocationForm.spec	✅ Hecha
HU023 Eliminación de cuenta	cuenta	DELETE /me (anonimiza, D12)	SecuritySettingsView · DeleteAccountModal	AccountTest · DeleteAccountModal.spec	✅ Hecha
HU024 Reporte de usuarios	calificaciones	POST /reports · GET /admin/reports · POST /admin/reports/{id}/resolve	ReportUserModal · AdminReportsView	UserReportsAndModerationTest · ReportUserModal.spec	✅ Hecha
HU025 Visualización de disponibilidad del profesional	agenda	GET /professionals/{id}/slots · evento availability.changed	SlotPicker (en NewBookingView y Reprogramar)	ProfessionalSlotsTest · AvailabilityTest · SlotPicker.spec · dates.spec	✅ Hecha
HU026 Estado del servicio	reservas	GET /bookings/{id} · GET /bookings/{id}/history · evento booking.status-changed	BookingDetailView · BookingHistory	BookingHistoryTest · BookingHistory.spec · presence.spec	✅ Hecha
HU027 Confirmación de inicio de servicio	reservas	POST /bookings/{id}/start	ServiceProgressCard	BookingsTest · ServiceProgressCard.spec	✅ Hecha
HU028 Confirmación de finalización de servicio	reservas	POST /bookings/{id}/complete	ServiceProgressCard	BookingsTest · ServiceProgressCard.spec	✅ Hecha
HU029 Visualización de comentarios	calificaciones	GET /professionals/{id}/reviews	ProfessionalDetailView (comentarios)	UserReportsAndModerationTest · ReviewsTest	✅ Hecha
HU030 Filtro por disponibilidad inmediata	búsqueda	GET /professionals?available_now=1	SearchResultsView (filtro de disponibles ahora)	UserReportsAndModerationTest	✅ Hecha
HU031 Edición de perfil del cliente	cuenta	PATCH /me	ProfileSettingsView	AccountTest · account/schemas.spec	✅ Hecha

HU	Módulo	Backend (API y eventos)	Frontend	Pruebas	Estado
HU032 Subida de foto de perfil	cuenta	POST /me/avatar	ProfileSettingsView	AccountTest	✅ Hecha
HU033 Visualización de servicios por categoría	búsqueda	GET /categories · GET /professionals?category_id=	SearchResultsView (categoría)	CatalogTest · ProfessionalsTest	✅ Hecha
HU034 Guardar profesionales favoritos	favoritos	POST/DELETE /favorites/{professionalId}	FavoriteButton	FavoritesTest · FavoriteButton.spec	✅ Hecha
HU035 Visualización de favoritos	favoritos	GET /favorites	MyFavoritesView	FavoritesTest · FavoriteButton.spec	✅ Hecha
HU036 Confirmación de reserva por el cliente	reservas	POST /bookings/{id}/confirm	ConfirmBookingButton	BookingsTest · ConfirmBookingButton.spec	✅ Hecha
HU037 Visualización de ingresos del profesional	perfil-profesional	GET /professional/earnings?from=&to=	EarningsView	EarningsTest · EarningsView.spec · ranges.spec	✅ Hecha
HU038 Bloqueo de usuarios problemáticos	verificación-admin	GET /admin/users · POST /admin/users/{id}/block · /unblock · middleware not_blocked	AdminUsersView	UserReportsAndModerationTest	✅ Hecha
HU039 Búsqueda por nombre	búsqueda	GET /professionals?q=	SearchResultsView (barra de búsqueda)	ProfessionalsTest	✅ Hecha
HU040 Visualización de distancia	búsqueda	GET /professionals?lat=&lng= (distance_km)	ProfessionalCard · ProfessionalDetailView	ProfessionalsTest · ProfessionalCard.spec	✅ Hecha
HU041 Indicador de estado en línea	chat-notificaciones	Canal de presencia booking.{id}	BookingChat (En línea / Desconectado)	BroadcastingTest · presence.spec	✅ Hecha
HU042 Adjuntar archivos en el chat	chat-notificaciones	POST /bookings/{id}/messages (attachment)	BookingChat	ChatTest · BookingChat.spec	✅ Hecha
HU043 Eliminación de mensajes	chat-notificaciones	DELETE /messages/{id} · evento message.deleted	BookingChat	ChatTest · BookingChat.spec	✅ Hecha
HU044 Visualización de perfil del cliente	calificaciones	GET /clients/{id} (solo el profesional con reserva)	ClientProfileModal (detalle de la reserva)	UserReportsAndModerationTest	✅ Hecha
HU045 Alerta por baja calificación	calificaciones	GET /admin/alerts · POST /admin/alerts/{id}/resolve	AdminAlertsView	UserReportsAndModerationTest	✅ Hecha
HU046 Ordenamiento por precio	búsqueda	GET /professionals?sort=price_asc	price_desc	SearchResultsView	ProfessionalsTest
HU047 Visualización de tiempo estimado	perfil-profesional	/professional/services (estimated_duration_minutes)	MyServicesView · NewBookingView · BookingDetailView	ProfessionalProfileTest · professional-profile/schemas.spec	✅ Hecha
HU048 Validación de datos obligatorios	auth	FormRequest → 422 con errores por campo	Formularios con VeeValidate + Zod	ApiErrorsTest · forms.spec · auth/schemas.spec	✅ Hecha

HU	Módulo	Backend (API y eventos)	Frontend	Pruebas	Estado
HU049 Visualización de servicios en proceso	reservas	GET /bookings?status=in_progress	InProgressPanel (paneles de inicio)	BookingsTest · InProgressPanel.spec	✅ Hecha
HU050 Cierre de sesión por inactividad	cuenta	— (cierre por inactividad en el cliente)	useInactivityTimer (App.vue)	Sin prueba automática; verificado a mano	✅ Hecha

Cobertura por módulo

Módulo	Responsable	Historias	Hechas
Autenticación (fase 0)	Equipo	HU001, HU006, HU007, HU021, HU048	5 de 5
Favoritos (fase 0)	Equipo	HU034, HU035	2 de 2
Agenda	LordCasta	HU003, HU025	2 de 2
Reservas	LordCasta	HU004, HU012, HU016, HU017, HU026, HU027, HU028, HU036, HU049	9 de 9
Perfil profesional	LordCasta	HU009, HU037, HU047	3 de 3
Chat y notificaciones	LordCasta	HU011, HU018, HU041, HU042, HU043	5 de 5
Búsqueda	barenas12	HU002, HU010, HU030, HU033, HU039, HU040, HU046	7 de 7
Cuenta	barenas12	HU019, HU020, HU022, HU023, HU031, HU032, HU050	7 de 7
Verificación y administración	barenas12	HU008, HU013, HU014, HU038	4 de 4
Calificaciones	barenas12	HU005, HU024, HU029, HU044, HU045	5 de 5
Total			**49 de 49**

La única historia sin prueba automatizada es la HU050 (cierre de sesión por inactividad), que vive solo en el cliente y se verificó a mano.

Plan y reporte de pruebas

1. Objetivo

Verificar de forma automática y repetible que cada historia de usuario cumple sus criterios de aceptación, que las reglas de negocio (agenda, estados de la reserva, permisos) no se rompen al agregar funcionalidades, y que la calidad del código se mantiene.

2. Estrategia

Nivel	Qué se prueba	Herramienta	Dónde
Pruebas de API (integración)	Cada endpoint: caso feliz, validación (422), permisos (401/403), reglas de estado (409) y efectos (registros, notificaciones, eventos, archivos)	Pest (PHPUnit) sobre SQLite en memoria	Backend `tests/Feature`
Pruebas unitarias del frontend	Funciones puras: fechas en hora de Colombia, rangos, horario semanal, esquemas de validación, cliente HTTP, formato	Vitest	Frontend `__tests__`
Pruebas de componentes	Interacción principal de cada componente con datos: lo que se muestra, lo que se envía a la API y cómo se manejan los errores	Vitest + Vue Test Utils (API simulada)	Frontend `__tests__`
Análisis estático	Tipos, estilo, imports entre módulos y colores fuera del sistema de diseño	TypeScript (vue-tsc), ESLint, oxlint, Pint, Prettier y verificador de tokens	Ambos
Seguridad de dependencias	Vulnerabilidades conocidas en las librerías	`composer audit`, `npm audit`	CI
Pruebas manuales	Recorridos completos en el navegador con dos sesiones (cliente y profesional), tiempo real y responsive	Navegador	Antes de fusionar cada HU

Reglas de la estrategia:

- Toda historia nueva incluye sus pruebas en el mismo pull request. Sin pruebas no cumple la definición de terminado.
- Por endpoint, como mínimo: caso feliz, validación y permisos.
- Las pruebas no dependen de servicios externos: la API se simula en el frontend, y en el backend se simulan las notificaciones, los eventos y el almacenamiento (`Notification::fake`, `Event::fake`, `Storage::fake`).
- Las fechas se fijan en las pruebas que dependen de la hora (`Carbon::setTestNow`, `vi.useFakeTimers`), para que den el mismo resultado cualquier día.
- No se permite saltar, enfocar ni borrar pruebas para que la CI pase: el check **Guardas** lo detecta y lo marca en el PR.

3. Integración continua

Cada pull request a `main` ejecuta automáticamente, y GitHub no permite fusionar si algo falla:

Repositorio	Verificaciones obligatorias
Backend	Formato (Pint) y pruebas (Pest) en **PHP 8.2 y 8.5**; Guardas
Frontend	Lockfile, lint (oxlint, ESLint, tokens de diseño), formato, tipos, pruebas y build de producción; Guardas
Ambos (informativo)	Auditoría de dependencias

4. Reporte de resultados

Ejecución del 22 de octubre de 2026 sobre `main`, con las 49 historias terminadas:

Suite	Pruebas	Resultado	Aserciones	Duración
Backend (Pest)	220	✅ 220 aprobadas	992	≈ 23 s
Frontend (Vitest)	158 en 38 archivos	✅ 158 aprobadas	—	≈ 60 s (en Windows)

4.1 Backend: pruebas por archivo

Archivo	Qué cubre
BookingsTest	Crear reservas (horario, bloqueos, cruces, precio), aceptar o rechazar, confirmar, iniciar y finalizar, listados y filtros, resumen, notificaciones
ProfessionalProfileTest	Perfil, especialidades, servicios con duración, portafolio y permisos
AvailabilityTest	Horario semanal, descanso, bloqueos, cruces y evento de agenda
BookingRescheduleTest	Reprogramar y cancelar con sus reglas y notificaciones
ChatTest	Enviar texto e imágenes, chat cerrado, permisos, leídos y eliminar mensajes
ApiErrorsTest	Formato estándar de errores (404, 405, 422, 401, 403)
ProfessionalsTest	Búsqueda pública, filtros, orden, cercanía, comunas aledañas, próxima hora libre y privacidad del perfil
ProfessionalSlotsTest	Cálculo de horas libres
NotificationsTest	Listado, contador y marcar como leídas
EarningsTest	Ingresos por rango y validaciones
BookingHistoryTest	Historial de estados y evento de estado
Auth/*, RegisterClientTest	Registro, inicio y cierre de sesión, y recuperación de contraseña sin revelar qué correos existen
AccountTest	Editar datos y ubicación, foto, cambio de contraseña y eliminación de cuenta con anonimización
VerificationTest, AdminVerificationTest	Cargue y reemplazo de documentos, y revisión del administrador
ReviewsTest	Calificación recíproca y promedio
UserReportsAndModerationTest	Reportes, bloqueo de usuarios, alertas, perfil del cliente, comentarios públicos y "disponible ahora"
FavoritesTest, CatalogTest, BroadcastingTest, ApiDocsTest, DatabaseSeederTest	Favoritos, catálogos, autorización de canales, documentación de la API y datos de demostración

4.2 Frontend: pruebas por área

Área	Archivos
Reservas	BookingResponseActions, ConfirmBookingButton, ServiceProgressCard, RescheduleBookingModal, BookingHistory, BookingList, InProgressPanel
Agenda	SlotPicker, WeeklyScheduleEditor, weekly, dates
Chat y tiempo real	BookingChat, presence
Notificaciones	NotificationBell
Perfil profesional	schemas, ServiceFormModal, EarningsView, ranges
Cuenta	ChangePasswordForm, DeleteAccountModal, ManualLocationForm, schemas, PasswordRecovery, RegisterProfessionalForm
Verificación y administración	AdminVerifications, ReplaceDocumentModal, schemas
Calificaciones y reportes	ReviewBookingModal, ReportUserModal, ProfessionalCard
Base	router (guards), cliente HTTP, formularios, formato, tokens, verificador de colores, favoritos, esquemas de auth

5. Cómo ejecutar las pruebas

```
# Backend
php artisan test
vendor/bin/pint --test

# Frontend
npm run lint
npm run type-check
npx vitest run
npm run build
```

6. Pendiente

- **Cobertura de código medida** (porcentaje de líneas): activar `--coverage` en los dos repositorios y publicar el resultado en este reporte.
- **Pruebas de extremo a extremo** de los recorridos principales con un navegador automatizado (Playwright).
- **Prueba de carga** para el requisito RNF-REN-01, sobre el servidor de producción.
- **Auditoría de accesibilidad** (Lighthouse) para RNF-USA-04.
- **Prueba automatizada del cierre por inactividad** (HU050) y de las vistas del administrador (usuarios, reportes y alertas), que hoy se prueban a mano.

Despliegue y operación

Guía para poner Maestros a un clic en producción en un servidor Linux (por ejemplo AWS EC2 con Ubuntu 24.04) y operarlo. La topología está en 03-diagramas.md §8. Para el entorno de desarrollo, ver el README de cada repositorio.

1. Requisitos del servidor

Componente	Versión	Para qué
PHP-FPM	8.2 o superior (probado hasta 8.5) con `mbstring`, `pdo_mysql`, `intl`, `gd`, `zip`, `bcmath`	API Laravel
Composer	2	Dependencias del backend
Node.js	20 o superior	Compilar el frontend (puede hacerse en la CI y subir solo `dist/`)
MySQL	8	Base de datos
Nginx	Estable	Servidor web, TLS y proxy de WebSockets
Supervisor	—	Mantener vivos la cola y Reverb
Certbot	—	Certificado TLS de Let's Encrypt

Tamaño sugerido para el piloto: 2 vCPU y 2–4 GB de RAM (por ejemplo, `t3.small`), con 30 GB de disco.

2. Variables de entorno del backend (`.env`)

Variable	Producción
`APP_ENV` / `APP_DEBUG`	`production` / `false`
`APP_URL`	`https://api.<dominio>`. Se usa para armar las URLs de archivos y las firmadas
`APP_KEY`	Generada con `php artisan key:generate` (no se comparte)
`DB_*`	Usuario de MySQL exclusivo de la app, con contraseña fuerte
`QUEUE_CONNECTION`	`database` (o `redis` si se agrega)
`BROADCAST_CONNECTION`	`reverb`
`REVERB_APP_ID`, `REVERB_APP_KEY`, `REVERB_APP_SECRET`	Valores propios, no los de desarrollo
`REVERB_HOST` / `REVERB_PORT` / `REVERB_SCHEME`	`api.<dominio>` / `443` / `https` (lo que ve el navegador)
`REVERB_SERVER_HOST` / `REVERB_SERVER_PORT`	`127.0.0.1` / `8080` (donde escucha Reverb detrás de Nginx)
`MAIL_*`	Proveedor SMTP para la recuperación de contraseña
`SANCTUM_STATEFUL_DOMAINS`	No aplica: la API usa tokens Bearer

Y en el frontend (antes de compilar): `VITE_API_BASE_URL=https://api.<dominio>/api/v1`, más `VITE_REVERB_APP_KEY`, `VITE_REVERB_HOST`, `VITE_REVERB_PORT=443` y `VITE_REVERB_SCHEME=https`.

3. Instalación del backend

```
cd /var/www/maestros-backend
git clone https://github.com/LordCasta/maestros_a_un_clic_backend.git .
composer install --no-dev --optimize-autoloader
cp .env.example .env    # y completar las variables de la tabla
php artisan key:generate
php artisan migrate --force
php artisan db:seed --class=CommuneSeeder --force   # comunas y corregimientos de Medellín
php artisan db:seed --class=CategorySeeder --force  # catálogo de categorías
# Nunca DemoSeeder ni DatabaseSeeder en producción: crean usuarios de demostración.
php artisan storage:link
php artisan config:cache && php artisan route:cache && php artisan event:cache
chown -R www-data:www-data storage bootstrap/cache
```

4. Frontend

```
npm ci && npm run build      # genera dist/
```

Copia `dist/` a `/var/www/maestros-frontend`. Nginx debe devolver `index.html` en cualquier ruta, porque el enrutamiento lo hace Vue.

5. Nginx

```
# Frontend
server {
    server_name <dominio>;
    root /var/www/maestros-frontend;
    location / { try_files $uri $uri/ /index.html; }
}

# API y WebSockets
server {
    server_name api.<dominio>;
    root /var/www/maestros-backend/public;
    index index.php;
    client_max_body_size 6M;           # imágenes de hasta 5 MB

    location / { try_files $uri $uri/ /index.php?$query_string; }
    location ~ \.php$ {
        include fastcgi_params;
        fastcgi_pass unix:/run/php/php8.3-fpm.sock;
        fastcgi_param SCRIPT_FILENAME $realpath_root$fastcgi_script_name;
    }
    # Reverb (WebSockets)
    location /app {
        proxy_pass http://127.0.0.1:8080;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "Upgrade";
        proxy_set_header Host $host;
        proxy_read_timeout 60s;
    }
}
```

Después: `certbot --nginx -d <dominio> -d api.<dominio>` para el TLS.

6. Procesos permanentes (Supervisor)

```
[program:maestros-queue]
command=php /var/www/maestros-backend/artisan queue:work --sleep=1 --tries=3 --max-time=3600
user=www-data
autostart=true
autorestart=true
stdout_logfile=/var/log/maestros/queue.log

[program:maestros-reverb]
command=php /var/www/maestros-backend/artisan reverb:start --host=127.0.0.1 --port=8080
user=www-data
autostart=true
autorestart=true
stdout_logfile=/var/log/maestros/reverb.log
```

Y en el `crontab` de `www-data`: `* * * * * php /var/www/maestros-backend/artisan schedule:run >> /dev/null 2>&1`.

7. Actualizar a una versión nueva

```
php artisan down
git pull origin main
composer install --no-dev --optimize-autoloader
php artisan migrate --force
php artisan config:cache && php artisan route:cache && php artisan event:cache
php artisan queue:restart          # los workers cargan el código nuevo
supervisorctl restart maestros-reverb
php artisan up
```

Las migraciones solo agregan cambios, nunca editan las existentes (regla del proyecto), así que se pueden aplicar sin perder datos.

8. Respaldos y restauración

Qué	Cómo	Frecuencia	Retención
Base de datos	`mysqldump --single-transaction maestros > backup-$(date +%F).sql`, comprimido	Diario (cron)	7 días en el servidor y copia fuera de él (por ejemplo, S3)
Archivos	`storage/app/public` y `storage/app/private` (portafolio, avatares, documentos y adjuntos)	Diario	7 días

Restaurar: `mysql maestros < backup-AAAA-MM-DD.sql` y copiar de vuelta las carpetas de `storage/app`. Conviene probar una restauración al menos una vez antes del lanzamiento.

9. Monitoreo

- **Salud:** `GET https://api.<dominio>/up` responde 200 si la app arranca. Configurar un monitor externo gratuito que lo consulte cada 5 minutos (RNF-DIS-01).
- **Errores:** `storage/logs/laravel.log`, con rotación diaria (`LOG_CHANNEL=daily`).
- **Cola:** `php artisan queue:failed` lista los trabajos fallidos; `queue:retry all` los reintenta.
- **Tiempo real:** si las notificaciones no llegan, revisa `supervisorctl status` y `/var/log/maestros/reverb.log`.

10. Lista de verificación antes de lanzar

- [ ] `APP_DEBUG=false` y `APP_ENV=production`.
- [ ] HTTPS activo en el frontend y en la API; WebSockets por `wss`.
- [ ] CORS limitado al dominio del frontend (publicar `config/cors.php`, RNF-SEG-08).
- [ ] Claves de Reverb y de la app propias, no las de desarrollo.
- [ ] Usuario de MySQL con permisos solo sobre su base.
- [ ] Respaldos programados y una restauración probada.
- [ ] Monitor de `/up` activo.
- [ ] Política de tratamiento de datos publicada y aceptada en el registro (RNF-PRI-03).

Seguridad y privacidad

Controles de seguridad implementados, cumplimiento del tratamiento de datos personales (Ley 1581 de 2012) y acciones pendientes. Se toma como referencia el **OWASP Top 10 (2021)**.

1. Datos que trata el sistema

Dato	Titular	Finalidad	Dónde se guarda	Quién lo ve
Nombre, correo, teléfono	Cliente y profesional	Cuenta y contacto	`users`	El propio usuario. El nombre lo ven además las contrapartes y, en el caso del profesional, el público
Comuna, dirección, coordenadas	Cliente y profesional	Búsqueda por cercanía y dirección del servicio	`users` , `bookings`	El propio usuario. La dirección de una reserva, solo sus partes
Selfie, documento de identidad, certificados	Cliente y profesional	Verificación de identidad (KYC)	Disco **privado**	El propio usuario y los administradores, con URL firmada temporal
Fotos del portafolio y foto de perfil	Profesional / usuario	Mostrar su trabajo y su identidad	Disco público	Cualquiera (es su finalidad)
Mensajes e imágenes del chat	Partes de la reserva	Coordinar el servicio	`messages` y disco **privado**	Solo las dos partes de la reserva
Calificaciones y comentarios	Cliente y profesional	Reputación	`reviews`	Público, en el perfil
Historial de reservas	Partes	Trazabilidad e ingresos	`bookings` , `booking_status_changes`	Las dos partes y los administradores

2. Controles frente al OWASP Top 10

Riesgo	Control en el sistema
A01 Control de acceso roto	Middleware de rol y Policies por recurso en cada acción (dueño, partes de la reserva); el perfil de un cliente solo lo ve el profesional con quien tiene una reserva; los usuarios bloqueados no acceden. Pruebas de 401 y 403 por endpoint
A02 Fallas criptográficas	Contraseñas con bcrypt; HTTPS y `wss` en producción; URLs firmadas con la clave de la app para los archivos privados
A03 Inyección	Consultas con el ORM y parámetros enlazados; validación de toda entrada con FormRequest; Vue escapa el contenido al mostrarlo (el chat no interpreta HTML)
A04 Diseño inseguro	Reglas de negocio centralizadas en servicios: máquina de estados, bloqueo de la agenda en una transacción para evitar dobles reservas, chat cerrado al terminar la reserva
A05 Configuración insegura	`APP_DEBUG=false` en producción, errores con formato estándar sin trazas, lista de verificación de despliegue. **Pendiente:** limitar CORS al dominio del frontend
A06 Componentes vulnerables	Auditoría de dependencias en cada pull request
A07 Fallas de autenticación	Tokens de Sanctum (se revocan al cerrar sesión, al restablecer la contraseña, al bloquear la cuenta y al eliminarla), máximo 10 intentos por minuto en login, registro y recuperación, contraseña de 8 caracteres o más, recuperación que no revela qué correos existen y cierre de sesión tras 15 minutos de inactividad (HU050). **Pendiente:** vencimiento de los tokens en el servidor (`sanctum.expiration`)
A08 Integridad de datos	CI obligatoria y check Guardas; las migraciones existentes no se editan; historial de estados auditable
A09 Registro y monitoreo	Log de la app con rotación diaria, historial de cambios de estado con autor y fecha, monitoreo de salud (`/up`)
A10 SSRF	El servidor no hace peticiones a URLs que entregue el usuario

3. Cumplimiento de la Ley 1581 de 2012

Principio o derecho	Cómo se atiende	Estado
Finalidad	Cada dato tiene una finalidad declarada (tabla de la sección 1)	✅

Principio o derecho	Cómo se atiende	Estado
Seguridad	Controles de la sección 2; documentos y adjuntos en almacenamiento privado	✅
Acceso y circulación restringida	El perfil público no muestra datos de contacto; el chat solo lo ven las partes	✅
Libertad (autorización previa)	Casilla obligatoria de aceptación de la *Política de tratamiento de datos* en el registro, con la fecha y la versión aceptadas	⚠️ Pendiente
Transparencia	Publicar la política (finalidades, derechos, canal de contacto del responsable)	⚠️ Pendiente
Consulta y rectificación	El usuario ve y edita sus datos, ubicación y foto desde su cuenta (HU009, HU022, HU031, HU032)	✅
Supresión	Eliminación de cuenta con anonimización: se borran datos de contacto, ubicación, fotos, documentos de verificación, favoritos y sesiones, y se conservan las reservas, reseñas y chats de la otra parte con el nombre "Usuario eliminado" (HU023, D12)	✅

4. Acciones pendientes

#	Acción	Responsable sugerido	Prioridad
1	Política de tratamiento de datos y aceptación obligatoria en el registro (campo `privacy_accepted_at` en una migración nueva)	Módulo cuenta	Alta
2	Limitar CORS al dominio del frontend en producción	Despliegue	Alta
3	Vencimiento de los tokens en el servidor (`sanctum.expiration`); el cierre por inactividad del navegador ya está (HU050)	Módulo cuenta	Media
4	Límite de frecuencia en el envío de mensajes del chat	Chat	Baja

Decisiones de arquitectura

Registro de las decisiones técnicas importantes (*Architecture Decision Records*): qué se decidió, por qué y qué consecuencias tiene. Las decisiones del modelo de datos (D1–D15) y las de negocio (N1–N3) están detalladas en docs/modelo-de-datos.md §1 y §6; aquí se resumen junto con las de arquitectura.

ADR-01 · Dos repositorios: backend y frontend

- **Decisión:** el backend (Laravel) y el frontend (Vue) viven en repositorios separados, con su propia CI, comunicados solo por la API.
- **Motivo:** responsabilidades claras, despliegues independientes y una forma de organizarse que el equipo considera más ordenada.
- **Consecuencia:** el contrato de la API (`docs/api/endpoints.md` y OpenAPI) es la frontera entre los dos, y un cambio que toca los dos lados se hace con un PR en cada repositorio: primero el backend.

ADR-02 · API REST versionada con un formato de respuesta único

- **Decisión:** API bajo `/api/v1`; toda respuesta tiene la forma `{ success, message, data, meta? }` y todo error, `{ success: false, message, errors? }`.
- **Motivo:** el frontend maneja todas las respuestas igual; una versión futura (`/v2`) no rompe a los clientes existentes.
- **Consecuencia:** los controladores solo responden con los helpers del controlador base, y los errores se formatean en un único lugar (`bootstrap/app.php`).

ADR-03 · Autenticación con tokens (Laravel Sanctum)

- **Decisión:** token Bearer por sesión, que se revoca al cerrar sesión.
- **Motivo:** la SPA y la API están en dominios distintos; los tokens evitan la complejidad de las cookies entre dominios y sirven también para autenticar los canales de WebSockets.

ADR-04 · Arquitectura por capas en el backend, sin patrón Repository

- **Decisión:** Ruta (middleware de rol) → FormRequest (validación) → Controller → Policy (autorización por recurso) → Service (reglas, transacciones, archivos) → Resource (forma de la respuesta). Las consultas reutilizables son *scopes* del modelo.
- **Motivo:** cada capa tiene una sola responsabilidad y las reglas de negocio quedan en un lugar fácil de probar. Un Repository sobre Eloquent agregaría una capa sin beneficio en este tamaño de proyecto.

ADR-05 · Frontend por módulos con API pública

- **Decisión:** `src/modules/<módulo>` autocontenidos (vistas, componentes, API, queries, rutas), que se usan entre sí solo por su `index.ts`; `src/shared` para lo común y `src/app` para el armado.
- **Motivo:** permite que dos personas trabajen en paralelo por módulos sin pisarse. ESLint bloquea los imports a archivos internos de otro módulo.

ADR-06 · Estado del servidor con Pinia Colada

- **Decisión:** los datos de la API se manejan como consultas con caché (Pinia Colada); Pinia solo guarda la sesión y los avisos.
- **Motivo:** evita duplicar el estado del servidor en *stores*. Invalidar una clave refresca todas las pantallas que la usan, que es lo que aprovechan los eventos de tiempo real.

ADR-07 · Tiempo real con Laravel Reverb (decisión N1)

- **Decisión:** WebSockets propios con Reverb: un canal privado por usuario para las notificaciones, uno de presencia por reserva para el chat, el estado y quién está conectado, y uno público por profesional para su agenda.
- **Motivo:** chat, estado en línea y agenda en vivo sin depender de un servicio de pago. Es compatible con el protocolo de Pusher, así que se usa Laravel Echo en el cliente.
- **Consecuencia:** hacen falta dos procesos más en producción: la cola y Reverb.

ADR-08 · Disponibilidad calculada, no guardada (decisión D7)

- **Decisión:** se guardan el horario semanal y los bloqueos; las horas libres se calculan al consultarlas, descontando las reservas activas y el descanso.
- **Motivo:** evita mantener miles de "cupos" sincronizados. Una reserva nueva valida los cruces dentro de una transacción que bloquea la agenda del profesional, para que dos clientes no tomen la misma hora.

ADR-09 · Máquina de estados y auditoría de la reserva (decisión D8)

- **Decisión:** los siete estados y sus transiciones válidas están en el enum `BookingStatus`; todo cambio pasa por un único método que valida, registra el historial y emite los eventos.
- **Motivo:** un solo lugar garantiza que nunca haya transiciones inválidas y que todo cambio quede auditado.

ADR-10 · Archivos privados con URL firmada (decisión D9)

- **Decisión:** los documentos de identidad y los adjuntos del chat van a un disco privado y se entregan con URLs firmadas que vencen; el portafolio y los avatares son públicos.
- **Motivo:** Ley 1581 de 2012. Nadie puede ver una cédula o una foto del chat con solo adivinar la URL.

ADR-11 · Sistema de diseño con tokens obligatorios

- **Decisión:** la paleta, la tipografía y las sombras se definen como tokens de Tailwind; un verificador en la CI rechaza los colores escritos a mano.
- **Motivo:** una identidad visual consistente aunque la construyan dos personas y asistentes de IA.

ADR-12 · Calidad protegida (CI obligatoria y check Guardas)

- **Decisión:** `main` está protegida. Solo se fusiona por PR, con todas las verificaciones en verde. El check Guardas falla si un PR modifica las reglas de calidad (lint, CI, configuración de pruebas, instrucciones para agentes) o una migración existente, salvo que una persona lo apruebe con la etiqueta `cambio-de-reglas`.
- **Motivo:** el equipo trabaja con asistentes de IA, que tienden a "arreglar" un fallo relajando la regla en lugar del código. Así eso queda bloqueado.

ADR-13 · Precio y duración por servicio (decisiones D6 y N3)

- **Decisión:** el profesional publica servicios con precio (fijo o por hora) y duración estimada. Al reservar se copia el precio a la reserva.
- **Motivo:** la duración define cuánto ocupa la reserva en la agenda, y un cambio de precio posterior no altera las reservas ya hechas.

ADR-14 · Verificación obligatoria para reservar (decisión N2)

- **Decisión:** cualquiera puede registrarse y explorar, pero solo un cliente con la identidad verificada puede crear reservas.
- **Motivo:** proteger a los profesionales, que van a la casa del cliente, sin frenar el registro.